Low Complexity and Mobility-Aware Robust Radio, Storage, Computing, and Cost Management for Cellular Vehicular Networks

Mohammad Reza Abedi, *Student Member, IEEE*, Nader Mokari, *Senior Member, IEEE*, Mohammad Reza Javan, *Senior Member, IEEE*, Hamid Saeedi, *Senior Member, IEEE*, Eduard A. Jorswieck, *Fellow, IEEE*, and Nizar Zorba, *Senior Member, IEEE*

***Abstract*—In this paper, a novel two-time scale task offloading/delivery framework is presented which dynamically manages radio, storage, computing, and cost aspects through multiple Mobile Edge Computing (MEC) servers distributed across Road Side Units (RSUs), Base Stations (BSs), vehicles, and pedestrians. To enhance flexibility, we introduce a smart dynamic task partitioning and cooperative offloading/delivery mechanism, enabling users to offload task partitions to various MEC servers. Uncertainties stemming from users' high mobility and traffic variations may lead to unpredictable task offloading demands and Imperfect Channel State Information (I-CSI), potentially causing unsuccessful task offloading/delivery. In response, we propose a robust and cooperative two-time-scale task offloading/delivery approach, adept at handling demand uncertainty and I-CSI. The problem formulation involves a two-time scale Markov decision process, aimed at minimizing task offloading/delivery costs while maximizing task completion rates. Large time-scale slots incorporate intelligent RSU activation/deactivation, while short time-scale slots focus on task offloading/delivery. To efficiently implement this framework, we propose Federated Reinforcement Learning (FRL) algorithm based on Deep Deterministic Policy Gradient (DDPG); namely Low overhead Multi-Agent DDPG (LoMADDPG) where at each slot, all agents tune their Actor and Critic neural network weights with the help of Global Software-defined Networking (SDN) Controller (GSC) which acts as a global weight tuner. Simulation results illustrate that our proposed task offloading/delivery scheme can result in significant improvement in task completion rate and cost reduction.**

***Index Terms*—Mobile edge computing, two-time scale task offloading/delivery, RSU activation/deactivation, demand and channel uncertainties, task partitioning, task migration.**

Received 29 February 2024; revised 20 August 2024 and 10 September 2024; accepted 6 October 2024. Date of publication 15 October 2024; date of current version 14 February 2025. The work of Nader Mokari and Mohammad Reza Abedi was supported by Iran National Science Foundation (INSF) under Project 4020595. The work of Eduard A. Jorswieck was supported in part by the Federal Ministry of Education and Research (BMBF), Germany, through the Program of Sourverän, Digital, Vernetz Joint Project 6G-RIC under Grant 16KISK031. This work was supported by the Qatar Research Development and Innovation Council (QRDI) under Grant ARG01-0511-230129. The review of this article was coordinated by Prof. Zhaolong Ning. *(Corresponding author: Nader Mokari.)*

Mohammad Reza Abedi and Nader Mokari are with ECE Department, Tarbiat Modares University, Tehran 14115-111, Iran (e-mail: Mohammadreza_abedi@modares.ac.ir; nader.mokari@modares.ac.ir).

Mohammad Reza Javan is with the Faculty of Electrical Engineering, Shahrood University of Technology, Shahrood 3619995161, Iran (e-mail: javan@shahroodut.ac.ir).

Hamid Saeedi is with the College of Engineering and Technology, University of Doha for Science and Technology, Doha 24449, Qatar (e-mail: hamid.saeedi@udst.edu.qa).

Eduard A. Jorswieck is with the Institute of Communications Technology, TU Braunschweig, 38106 Braunschweig, Germany (e-mail: jorswieck@ifn.ing.tu-bs.de).

Nizar Zorba is with the College of Engineering, Qatar University, Doha 2713, Qatar (e-mail: nizarz@qu.edu.qa).

Digital Object Identifier 10.1109/TVT.2024.3480996

I. Introduction

WHEN the on-board computing power of vehicles can not satisfy the computing and power consumption requirements, Mobile Edge Computing (MEC) can improve Quality of Service (QoS) in terms of latency, reliability, and resource efficiency by providing computing, storage, and network resources closer to the network edge and devices. To achieve this, the tasks can be offloaded between vehicles and edge devices through Vehicle-to-everything (V2X) communication.

The uncertainties arising from the high mobility of vehicles further complicate matters, particularly demand uncertainty, where node task offloading demands are unknown and subject to change. Existing works often assume deterministic task offloading demands, leading to inefficient resource utilization and increased task offloading and delivery delays. This paper addresses the need for a more adaptive approach by considering task offloading under demand uncertainty, with each node's computational task offloading requests being probabilistic. The complexity of most wireless Resource Allocation (RA) optimization problems renders traditional heuristic algorithms inadequate, providing only sub-optimal solutions without performance guarantees. However, the rise of Machine Learning (ML) algorithms has shown remarkable success in RA, pattern recognition, and signal processing. Utilizing ML algorithms for real-time RA and navigation in vehicular networks holds a great promise for addressing these challenges effectively. More precisely, distributed multi-agent learning methods can help overcome demand uncertainty in wireless networks by leveraging the collective knowledge and data of multiple devices while minimizing the need for centralized coordination and communication. This approach is particularly useful in wireless networks where resource availability and demand can vary over time and space. Indeed, large overhead during training of ML algorithms can be problematic and this issue was not attended to in some previous works [1], [2].

0018-9545 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

In this paper, we consider the task offloading/delivery strategy in high mobility vehicular networks through adaptive radio, storage, computing, and cost management. We propose a two-time scale resource management scheme for task offloading in which we aim to minimize the total network cost and task delay for each task. It involves large time-scale Road Side Unit (RSU) activation/deactivation, as well as short time-scale task offloading/delivery and RA among subscribed users. Two major challenges including user traffic variation over time and Channel State Information (CSI) uncertainty are considered during RA and task offloading/delivery decisions. To tackle these challenges and minimize the total cost of the network and delay of each task, the interaction between Base Station (BS) activation/RA as well as RSU activation/RA is considered. The large time-scale RA and activation, characterizes the statistics of user traffic. It also ensures that sufficient RSUs are activated for short time-scale RA and task offloading decisions. This is achieved by taking into account CSI uncertainty to achieve efficient utilization of the predefined BSs and RSUs activation, and their resources to guarantee the QoS. In the short time-scale RA, the RA and task offloading decisions are performed based on given decisions from the large time-scale stage stage, which is adaptive to real-time user task offloading request and delivery.

Consequently, in this paper we propose a cooperative robust two-time scale task offloading scheme in high mobility vehicular networks and compare it with its deterministic counterpart scheme as a benchmark. Since our optimization problem is non-convex and NP-hard, we adopt ML-based algorithms to solve it efficiently. Furthermore, we extend our problem as two-time scale Markov Decision Process (MDP) since we have different time scales; slow and fast. In other words, to address the challenge of multi-time-scale operations, we have introduced an innovative deep reinforcement learning (DRL) algorithm that allows the coordination of operations on different time scales to work together to achieve the desired system goal. Then, we propose Low overhead Multi-Agent Deep Deterministic Policy Gradient (LoMADDPG), a Federated Reinforcement Learning (FRL) based algorithm, in a two-time scale fashion. Eventually, we formulate a deterministic optimization problem and then extend it into a robust scheme by considering uncertainty on traffic demands and channel estimation. The major novelty and main contributions of our work are summarized as follows:

- Our two-time scale task offloading scheme enhances efficiency in high mobility vehicular networks by strategically controlling RSU activation during large-scale time slots, while users offload tasks with RSUs and BSs during short-scale time slots.
- To further increase flexibility, we consider task partitioning, which allows each user's task to be divided into partitions, enabling offloading to different MEC servers. Furthermore, we address the challenge of high mobility, leading to unsuccessful offloading, by introducing a task migration scheme that allows migration from unreliable MEC servers to more reliable nodes to improve task completion rates. Our proposed framework accounts for uncertainty in task offloading demand and CSI estimation to achieve robustness.
- We introduce LoMADDPG, an FRL using a DDPG agent, to solve the two-time scale optimization problem in vehicular networks. Our method improves resource allocation and navigation efficiency. We compare it with No-Partition and No-Migration cases and find that LoMADDPG reduces task offload costs and communication overhead, outperforming MADDPG in task completion rate even in uncertain conditions.

The rest of this paper is organized as follows: In the next section, we review the related works. Section III describes our system model. In Section IV, we present the problem formulation for our problem. In Section IV-C, we present the proposed multi-agent DRL based RA scheme and computing strategy design. Our experimental results are provided in Section V. Finally, Section VI concludes this paper.

II. Related Works

Authors in [3] directed their attention toward vehicular fog-edge computing and delved into incentive-driven computational offloading and resource allocation strategies for vehicular networks utilizing 5G-NR V2X communications. The primary aim is to optimize the comprehensive utility function of the system, taking into account elements such as delay and incentive fulfillment, all while adhering to set deadlines and incentive constraints. Notably, given the considerably smaller scale of partitioned tasks in our proposed approach, this logically results in the identification of a larger number of MEC servers positioned optimally to handle a fraction of the task's processing. This cascade effect leads to a reduction in both power consumption and delays, thereby fostering an increase in the volume of processed tasks that align with the desired quality standards throughout the entire network. In the literature, two primary approaches have been proposed to tackle the movement of users between MEC hosts while making use of MEC services. The first approach involves maintaining the operational MEC service on the original MEC host, while the second approach entails transferring the ongoing service to the MEC host linked with the new RSU. Although the former approach is simpler in terms of service reconfiguration and migration, it had the potential to lead to considerable overhead in scenarios marked by frequent handovers, like situations with highly mobile vehicular terminals. Furthermore, latency could experience a significant rise when MEC applications are not active on the MEC hosts of the current RSUs but rather on those of other RSUs. Furthermore, the migration of MEC services itself constituted a costly procedure that could disrupt these services. In response to these concerns, certain earlier studies have put forth hybrid methodologies, which merge MEC service migrations with the retention of specific services on the preceding MEC hosts [4]. In [5], the authors implement infrastructure-based camera sensing with 5G and MEC computing for connected safety applications. They create the Raven platform for this purpose and developed three safety applications: Intersection T-Bone Vehicle Crash Warning, Vehicle-in-blind-spot Warning, and Stopped/Disabled Hazard Vehicle Warning. Raven uses a roadside camera connected to an MEC server via 5G to monitor vehicles, calculate dynamics, and transmit data to subscribed vehicles. Most preceding

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

research [3], [6] assume that the user holds the responsibility for accomplishing a segment of the task, leaving the remaining part to be forwarded to the MEC server. However, this strategy frequently leads to incomplete task execution due to inadequate resources available at the assigned MEC server. In contrast, our underlying assumption is that a task fraction earmarked for MEC server processing could be proficiently subdivided into multiple optimal segments and then distributed to diverse MEC servers through an optimization challenge. This methodology bears the potential to considerably elevate the processing velocity of these tasks.

In [7], the aim is to reduce the response time and energy usage by solving a combinatorial optimization problem for urban MEC-enabled vehicular networks by taking into account vehicle mobility. A task priority determination scheme enhances system stability and offloading success rate by prioritizing tasks in the waiting queue. To handle changes in CSI and network structure in vehicular networks, the authors in [8] suggest a novel solution that views task offloading as a multi-armed bandit problem and introduce an algorithm that allows vehicle nodes to learn about service provider delay performance during task offloading. The RSU plays a crucial role in facilitating information exchange among vehicle nodes, enhancing the algorithm's adaptability to the network's dynamic nature. In [9], the authors examine energy-efficient computation offloading in vehicular edge computing with speed-adjustable vehicles to meet latency constraints and enhance the transportation system's collaborative capabilities. To overcome the limitations of assigning tasks to a single computing node, task replication is used to distribute tasks to multiple nodes simultaneously in [10]. In [11], the authors propose a model to address unreliable vehicular communications in task delegation. They develop a bi-objective optimization problem called the Partitioning and Offloading Problem, using Deep Neural Networks (DNNs) to maximize acceleration and service ratios. To solve this problem, the authors propose a Distributed Partitioning and Offloading Solution, which includes a delay-priority-oriented offloading strategy for informed decision-making by edge nodes and a stacking-based partitioning strategy to assist client vehicles in effective partitioning decisions. In [12], the authors examine task offloading problem in a cooperative MEC network. They formulate it as a convex constrained optimization problem, considering both linear and nonlinear inequality constraints such as service latency, energy consumption, and a combination of both. To solve this problem, the authors develop a distributed algorithm based on gradient projection and virtual queue techniques. In [13], the authors propose a new method for optimizing resources in a multi-dimensional setting using a MDP. Their goal is to minimize system delay and cache cost over time. To handle the problem's two-timescale nature, they introduce a centralized dual-actor DDPG algorithm. This algorithm uses a dual-actor network to make resource management decisions for both long-term and short-term aspects, while a centralized critic network guides joint actions. In [14], the authors analyze an orchestrated Edge Crowd Computing (ECC) system where devices rent out resources for incentives. They propose the Incentive-Vacation Queueing (IVQ) model to associate performance with incentive payments. The authors present a framework to analyze sharing economy systems using the IVQ model, focusing on the relationship between incentives, vacations, and performance, and enhances understanding of how incentives and vacations impact the performance of an orchestrated ECC system. The authors goal in [15] is to optimize task offloading and resource allocation in MEC networks for efficient service request handling. However, the complexity of applications such as intelligent transportation and autonomous driving presents challenges in modeling and problem-solving. To address this, they have formulated a spatial-temporal correlative problem to minimize task completion delay and energy consumption by developing a double deep Q network (DDQN) for task offloading, integrating subcarrier allocation and transmit power control for long-term system performance optimization. In [16], the authors develop a Vehicular Edge Computing (VEC) system with a tailored task offloading algorithm for Internet of Vehicles (IoV), using a four-stage Stackelberg game and reinforcement learning model to analyze vehicle state, communication methods, and offloading strategy. In [17], the authors study mobility-aware computation offloading and resource allocation for NOMA MEC in vehicular networks to reduce task completion latency. The study addresses two sub-problems: general multicarrier NOMA communication resource allocation and joint computation offloading and resource allocation, solved using convex optimization theory. Vehicle Terminals (VTs) are categorized into two groups based on sojourn time constraints, and an algorithm is proposed for computation offloading and resource allocation for each group of VTs. In [18], the goal is to efficiently schedule subtasks with known dependencies in multi-vehicle situations, focusing on reducing system latency and energy consumption. A subtask priority scheduling approach is proposed, utilizing the Directed Acyclic Graph (DAG) topological structure to maintain the order of subtasks, particularly in scenarios with intricate interconnections.

In contrast to earlier studies, our suggested framework incorporates a mechanism in which each BS, relying on task traffic statistics, places the deployed MEC servers at RSUs in standby mode during periods of low traffic volume, with the objective of minimizing power consumption. Offloading and resource allocation tactics are frequently formulated without considering the distinctive attributes of the C-V2X link. Specifically, the vehicle sensing capabilities within the C-V2X architecture lack consistent reliability owing to the frequent shifts in network topology. Our proposed model integrates a partitioning and offloading approach that demonstrates resilience against fluctuations in traffic. Considering the swift movement of vehicles, modifications in task offloading requests and traffic patterns within cells are inevitable. Thus, the development of a model capable of withstanding such changes becomes imperative.

III. System Model

A. Overview

We consider an SDN-based and MEC-enabled cellular vehicular network consisting of V vehicles, P pedestrians, R

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

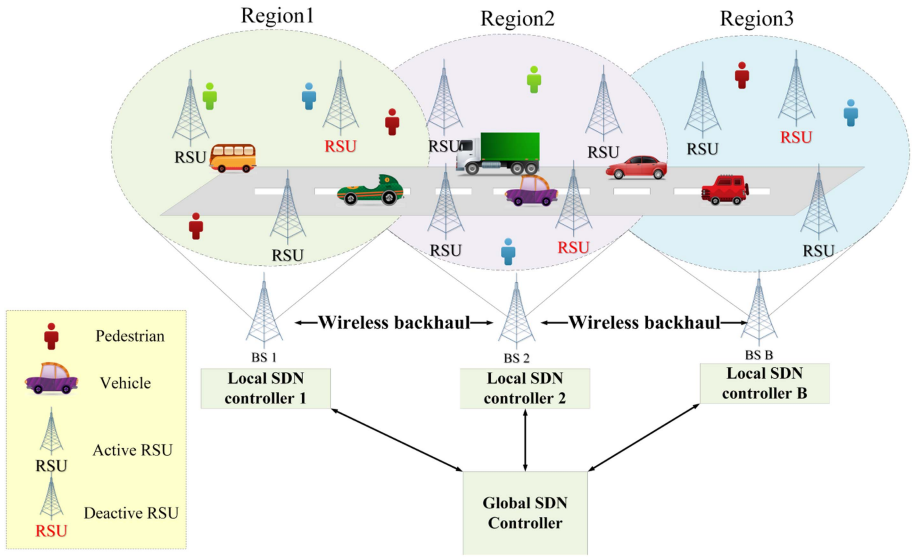


Fig. 1. A vehicular network consists of several vehicles, pedestrians, BSs, and RSUs. The BSs collaboratively support task offloading of vehicles and pedestrians while minimizing the network cost by deactivating some RSUs.

RSUs, and B BSs. All BSs are SDN-enabled, as shown in Fig. 1. Each BS is equipped with an intelligent Local SDN Controller (LSC) that controls its assigned RSUs, i.e., BS b controls its assigned RSUs, $\mathcal{R}_b = \{r_1, r_2, \ldots, r_{R_b}\}$, where $R_b = |\mathcal{R}_b|$ denotes the number of assigned RSUs to BS b. All LSCs are controlled by the Global SDN Controller (GSC). Let $\mathcal{V} = \{v_1, v_2, \ldots, v_V\}$, $\mathcal{P} = \{p_1, p_2, \ldots, p_P\}$, $\mathcal{R} = \{r_1, r_2, \ldots, r_R\}$, and $\mathcal{B} = \{b_1, b_2, \ldots, b_B\}$ denote the set of vehicles, pedestrians, RSUs, and BSs, respectively. Each node $i \in \mathcal{P} \cup \mathcal{V} \cup \mathcal{R} \cup \mathcal{B}$ has limited computational power $C_i^{\max}$, storage $M_i^{\max}$, and bandwidth $\omega_i^{\max}$. Due to their computational power and battery capacity limitations, the users, i.e., pedestrians and vehicles may not meet their delay constraint for processing their tasks locally. Consequently, users can offload their tasks to be executed on different nodes. However, due to their high mobility, they may encounter increased latency during the task offloading process or task delivery as they move farther away from the corresponding nodes. To maintain load balance and reduce energy consumption, the network may also migrate tasks from current nodes to others. Each BS and its serving RSUs are under control of an LSC. Each LSC is implemented in the corresponding BS. We assume that all RSUs, pedestrians, and vehicles are connected to the corresponding BS (e.g., LSC) via control plane communication. On the other hand, as demand for task offloading increases, the availability of resources, i.e., communication, computing, and storage resources of each RSU eventually becomes insufficient to support the required QoS for pedestrians and vehicles. Conversely, dense deployment of RSUs may result in waste of power and resources and imposes more cost to the network. Therefore, the intelligent LSC provides a balance between the task offloading QoS and the network cost, by turning some RSUs into standby or active modes (activate/deactivate) [19]. In other words, by increasing or decreasing the number of active RSUs and their resources, including transmit power and bandwidth, the LSC aims to mitigate the cost of the network, while guaranteeing all pedestrians and vehicles task offloading requirements.

B. Two-Time Scale Framework

We assume that the time is divided into two different time scales as short/large (equivalently fast/slow) time scales denoted as t_τ and τ, respectively, where each large time-scale slot τ consists of T short time-scale slots t_τ. RSU activation/deactivation and resource management follow a periodic pattern, forming a large time-scale slot τ. User traffic variations occur at different indices within these slots. The time horizon is divided into multiple large time-scale slots, each denoted by τ with a fixed duration Δ.

Each large time-scale slot is divided into T short time-scale slots, indexed by t_τ with a duration of δ ($\Delta = T\delta$, as illustrated in Fig. 2). The chosen duration for each large time-scale slot ensures that user traffic statistics remain constant within it. However, a shorter duration may lead to more frequent RSU activation/deactivation and RA, resulting in higher computation and reconfiguration costs. Therefore, the duration of each large time-scale slot should be chosen carefully to avoid excessive computation and reconfiguration expenses. The set of all short time-scale slots in a large time-scale slot is denoted by $\mathcal{T}$. At the beginning of each large time-scale slot τ, each LSC decides to activate/deactivate RSUs and determines their maximum transmit power and amount of bandwidth in order to satisfy the pedestrians and vehicles task offloading requirements with the statistical knowledge of user traffic. The decision is made based on traffic demands and the corresponding required resource capacity of pedestrians and vehicles, as well as their mobility, uncertainty on channel gains and traffic demands, position, task characteristics, and QoS requirements. Moreover, we consider cost $\phi_b(\tau)$ that is imposed to GSC when BS b is serving at decision time τ, which

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

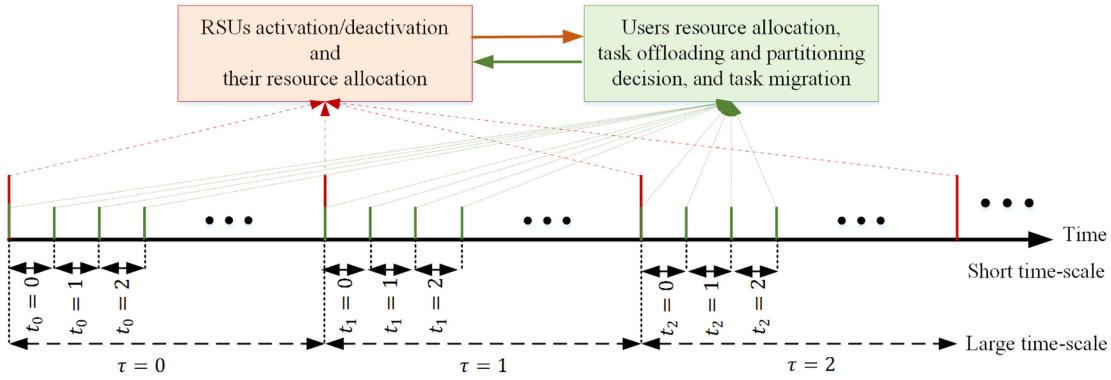


Fig. 2. Two-time scale activation/deactivation, RA and making task offloading and delivery decisions.

includes the fixed cost $\phi_b^0(\tau)$ and the cost of bandwidth, transmit power, and computing resources used at each BS, denoted by $\phi_b^{\text{BW}}(\tau)$, $\phi_b^{\text{Pow}}(\tau)$, and $\phi_b^{\text{Com}}(\tau)$, respectively. In addition, we consider the cost $\tilde{\phi}_{br}(\tau)$ that is imposed to LSC b when RSU r, assigned to BS b, is activated at decision time τ, which includes the fixed cost $\tilde{\phi}_{br}^0(\tau)$ and the cost of bandwidth, transmit power, and computing resources used at RSU r, denoted by $\tilde{\phi}_{br}^{\text{BW}}(\tau)$, $\tilde{\phi}_{br}^{\text{Pow}}(\tau)$, and $\tilde{\phi}_{br}^{\text{Com}}(\tau)$, respectively. We let $\tilde{\text{a}}_{br}(\tau)$ denote a binary variable with $\tilde{\text{a}}_{br}(\tau) = 1$ when RSU r is activated at decision time τ by LSC b and otherwise $\tilde{\text{a}}_{br}(\tau) = 0$. We also let $\tilde{\phi}_{b\bar{b}}^{\text{OverHead}}(\tau)$ denote the communication overhead cost between agents (e.g., LSCs) when an LSC transfers K bits data to other LSC on the backhaul link [20].

C. Mobility-Aware Task Offloading and Delivery

During each large time slot τ, the task offloading procedure commences within each short time-scale slot t_τ. The duration of each short time-scale slot is carefully chosen to capture real-time user traffic variations, allowing each RSU to dynamically allocate resources and handle arbitrary user requests. The durations of both the large and short time-scale slots remain fixed and do not change over time. Specifically, short time-scale slots have a duration of 125 μs, defined as one TTI, while large time-scale slots consist of 5000 TTI.

1) Task Offloading and Computation Scheme: At each short time-scale slot t_τ, each RSU r decides about task offloading and migration of its serving user's tasks. For the users, there are different tasks such as healthcare intensive tasks and dynamic ride sharing. We assume user $u \in \mathcal{P} \cup \mathcal{V}$ has a computation task described by $\mathcal{O}_u = \{z_u, \eta_u, D_u^{\max}\}$, where z_u is the size of task in bits for user u, η_u is the required computation resource for each bit in cycles for the task of user u, and $D_u^{\max}$ is the maximum tolerable delay in millisecond for task of user u. We adopt the task partition method for task offloading and delivery, where the task of user u can be divided into Q_u portions with $\mathcal{Q}_u = \{q_{1u}, q_{2u}, \ldots, q_{Q_u u}\}$ and then a portion can be processed locally and other parts can be sent to other nodes, i.e., BSs, RSUs, other vehicles and pedestrians. Let $\lambda_u^{q_u} \in [0,1]$ determine the sizes of the qth portion of task of user u as $q_{q_u u} = \lambda_u^{q_u} z_u$, where $\sum_{q_u \in \mathcal{Q}_u} \lambda_u^{q_u} = 1, \forall u$ and $\sum_{q_u \in \mathcal{Q}_u} q_{q_u u} = z_u$. Let $\check{\rho}_{u,b}^{q_u}(t_\tau)$ and $\rho_{ui,b}^{\kappa q_u}(t_\tau)$ indicate a binary decision variable, where $\check{\rho}_{u,b}^{q_u}(t_\tau) = 1$ and $\rho_{ui,b}^{\kappa q_u}(t_\tau) = 1$ if user u offloads the qth portion of its task to itself (e.g., locally process) or another nodes $i \in \mathcal{P} \cup \mathcal{V} \cup \mathcal{R} \cup \mathcal{B}, i \neq u$ controlled by RSU or BS[1] b on the relative subcarrier $\kappa \in \mathcal{N} \cup \tilde{\mathcal{N}}$ at short time-scale slot t_τ, respectively, otherwise $\check{\rho}_{u,b}^{q_u}(t_\tau) = 0$ and $\rho_{ui,b}^{\kappa q_u}(t_\tau) = 0$, where $\mathcal{N}$ and $\tilde{\mathcal{N}}$ denote the sets of subcarriers. In addition, let $\tilde{\rho}_{ui,b}^{\kappa q_u}(t_\tau)$ indicate a binary decision variable for relaying the qth portion of task of user u, where $\tilde{\rho}_{ui,b}^{\kappa q_u}(t_\tau) = 1$ if the qth portion of task of user u is only relayed by node i controlled by b on relative subcarrier κ at short time-scale slot t_τ, respectively; otherwise $\tilde{\rho}_{ui,b}^{\kappa q_u}(t_\tau) = 0$. Each user u must offload the qth portion of its task on only one node's server at short time-scale slot t_τ, which is ensured by the following constraints for all $i, j \in \{p, v, r, b\}, i \neq j, i \neq u$:

$$\text{C1: } \check{\rho}_{u,b}^{q_u}(t_\tau)\rho_{ui,b}^{\kappa q_u}(t_\tau) = 0, \text{C2: } \rho_{ui,b}^{\kappa q_u}(t_\tau)\rho_{uj,b}^{\kappa q_u}(t_\tau) = 0, \quad (1)$$

$$\text{C3: } \tilde{\rho}_{ui,b}^{\kappa q_u}(t_\tau)\tilde{\rho}_{uj,b}^{\kappa q_u}(t_\tau). \quad (2)$$

For successful task offloading and guaranteeing that any portion of task being processed for each user u, the following constraints must be met for all u, τ:

$$\text{C4: } \sum_{t_\tau \in \mathcal{T}} \sum_{q \in \mathcal{Q}_u} \sum_{\kappa \in \mathcal{N} \cup \tilde{\mathcal{N}}} \sum_{\substack{i \in \mathcal{P} \cup \mathcal{V} \cup \mathcal{R}, \\ i \neq u}} (\check{\rho}_{u,b}^{q_u}(t_\tau) + \rho_{ui,b}^{\kappa q_u}(t_\tau)) q_{q_u u} = z_u. \quad (3)$$

In addition, by the following constraints, we ensure that the utilization of memory and CPU at nodes do not exceed the available resources for all $i \in \mathcal{P} \cup \mathcal{V} \cup \mathcal{R} \cup \mathcal{B}$:

$$\text{C5: } \sum_{t_\tau \in \mathcal{T}} \sum_{u \in \mathcal{P} \cup \mathcal{V}} \sum_{b \in \mathcal{B}} \sum_{\kappa \in \mathcal{N} \cup \tilde{\mathcal{N}}} \sum_{q_u \in \mathcal{Q}_u} [\check{\rho}_{i,b}^{q_u}(t_\tau) + \rho_{ui,b}^{\kappa q_u}(t_\tau)] q_{q_u u} \leq M_i^{\max}, \quad (4)$$

$$\text{C6: } \sum_{t_\tau \in \mathcal{T}} \sum_{u \in \mathcal{P} \cup \mathcal{V}} \sum_{b \in \mathcal{B}} \sum_{\kappa \in \mathcal{N} \cup \tilde{\mathcal{N}}} \sum_{q_u \in \mathcal{Q}_u} [\check{\rho}_{i,b}^{q_u}(t_\tau) + \rho_{ui,b}^{\kappa q_u}(t_\tau)] q_{q_u u} \zeta_o^u \leq C_i^{\max}, \quad (5)$$

[1]For simplicity, in this section (Section III), we consider subscript b for both RSU and BS. In Section IV-C, we consider separate subscripts for each RSU and BS.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

where ζ_o^u denotes the CPU cycles required for processing one bit of task of user u. Moreover, by flow conservation constraint, we ensure that the qth portion of task of user u which is offloaded to node $i \in \{p, v, r, b\}$, leaves its source node u and eventually arrives at its destination node $i \in \{p, v, r, b\}$ [21] as follows:

$$\text{C7:} \sum_{i \in \mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{C}} (\rho_{ui,b}^{\kappa q_u}(t_\tau) + \tilde{\rho}_{ui,b}^{\kappa q_u}(t_\tau)) - \sum_{i \in \mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{C}} (\rho_{ui,b}^{\kappa q_u}(t_\tau) + \tilde{\rho}_{ui,b}^{\kappa q_u}(t_\tau)) = \begin{cases} 1 & \text{If } u \text{ is the source of portion } q \text{ of task,} \\ -1 & \text{If } u \text{ is determined to process portion } q \text{ of task,} \\ 0 & \text{otherwise.} \end{cases} \tag{6}$$

Similar process can be considered for the delivery phase.

2) Communication Links: We consider two different communication protocols as DSRC between vehicles and pedestrians with total bandwidth ω Hz and the C-V2X with total bandwidth $\tilde{\omega}$ Hz between BSs/RSUs and vehicles/pedestrians. For both communication protocols, the Orthogonal Frequency-division Multiple Access (OFDMA) technique is applied. Also, we consider that the whole bandwidth ω is uniformly divided into multiple N subcarriers denoted by n, each with bandwidth w_n and the whole bandwidth $\tilde{\omega}$ is divided into $\tilde{N}$ subcarriers indexed by $\tilde{n}$, each with bandwidth $\tilde{w}_{\tilde{n}}$. $\mathcal{N} = \{n_1, n_2, \ldots, n_N\}$ and $\tilde{\mathcal{N}} = \{\tilde{n}_1, \tilde{n}_2, \ldots, \tilde{n}_{\tilde{N}}\}$ denote the sets of subcarriers. We assume that all transceivers are equipped with a single antenna. We assume that node $i \in \mathcal{P} \cup \mathcal{V} \cup \mathcal{R} \cup \mathcal{B}$ communicates with node $j \in \mathcal{P} \cup \mathcal{V} \cup \mathcal{R} \cup \mathcal{B}, j \neq i$. For a coherence time period, the channel power gain between node i and node j over subcarrier κ, is given as $g_{ij}^{\kappa}(t) = \alpha_{ij} h_{ij}^{\kappa}(t)$, where $\alpha_{[.]}$ is the path loss of communication link and computed as $\alpha_{ij} = 10^{-\frac{63.3+17.7\log_{10}(d_{ij})}{10}}$, where d_{ij} is the distance between two nodes i and j, and $h_{ij}^{\kappa}(t)$ is the small-scale fading [22]. This model for path loss is provide for V2V communication based on different network parameters like center frequency f_c, bandwidth, complex normal distribution characterizes with a zero mean and unit variance etc. [23], [24]. In this path loss model, 63.3 is a constant that includes various factors such as system losses, antenna gains, and the frequency component. This specific value is determined based on a frequency of 3.5 GHz, which is the primary frequency for fifth-generation networks. 17.7 is a path loss exponent derived from empirical measurements indicating how path loss increases with distance. The constant term 63.3 typically encompasses the frequency component, so the model implicitly assumes a specific frequency value. Due to fast changing environment and the Doppler effect, we assume that there exists uncertainty in small-scale fading [25], [26], hence $h_{ij}^{\kappa}(t) = \bar{h}_{ij}^{\kappa}(t) + \epsilon_{ij}^{\kappa}(t)$, where $\bar{h}_{ij}^{\kappa}(t)$ and $\epsilon_{ij}^{\kappa}(t)$ are the estimated and error of channel power gains, respectively. In addition, the uncertainty region of the channel power gains can be defined as the distance between the channel power gains and its estimated value as follows:

$$\text{C8: } \mathfrak{R}_{ij}^{\kappa} = \{h_{ij}^{\kappa}(t) |\ |h_{ij}^{\kappa}(t) - \bar{h}_{ij}^{\kappa}(t)| \leq \varepsilon_{ij}^{\kappa}\}, \tag{7}$$

where $|\cdot|$ is the absolute value and $\varepsilon_{ij}^{\kappa}$ denotes the bound of the uncertainty region. Accordingly, the data rate between i and j controlled by b on subcarrier κ with bandwidth w_κ is calculated as follows:

$$\mathrm{r}_{ij,b}^{\kappa}(t_\tau) = \begin{cases} \tilde{\mathrm{r}}_{ij,b}^{\tilde{n}}(t_\tau) = w_{\tilde{n}} \log_2\left(1 + \tilde{\text{SINR}}_{ij,b}^{\tilde{n}}(t_\tau)\right), & \text{If } \kappa = \tilde{n}, \\ \check{\mathrm{r}}_{ij,b}^{n}(t_\tau) = w_n \log_2\left(1 + \check{\text{SINR}}_{ij,b}^{n}(t_\tau)\right), & \text{If } \kappa = n, \end{cases} \tag{8}$$

where $\tilde{\text{SINR}}_{ij,b}^{\tilde{n}}(t_\tau) = \frac{\tilde{p}_{ij,b}^{\tilde{n}}(t_\tau)\tilde{g}_{ij}^{\tilde{n}}(t_\tau)}{\tilde{I}_j^{\tilde{n}}(t_\tau)+\sigma_j^2}$ and $\check{\text{SINR}}_{ij,b}^{n}(t_\tau) = \frac{\check{p}_{ij,b}^{n}(t_\tau)\check{g}_{ij}^{n}(t_\tau)}{\check{I}_j^{n}(t_\tau)+\sigma_j^2}$. For these equations, $\tilde{p}_{ij,b}^{\tilde{n}}(t_\tau)$ and $\check{p}_{ij,b}^{n}(t_\tau)$ denote the transmission power of node i to node j at short time-scale slot t_τ on subcarriers n and $\tilde{n}$, respectively, σ_j^2 represents the power of the Gaussian noise in node j, and $\tilde{I}_j^{n}(t_\tau)$ and $\check{I}_j^{\tilde{n}}(t_\tau)$ are the interference from DSRC and C-V2X communications at j at short time-scale slot t_τ on subcarriers n and $\tilde{n}$, respectively, which are calculated as follows:

$$I_j^{\kappa}(t) = \sum_{\bar{i} \in \mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{C},\ \bar{i}\neq i} \ \sum_{\bar{j} \in \mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B},\ \bar{j}\neq j} p_{\bar{i}\bar{j}}^{\kappa}(t) g_{\bar{i}j}^{\kappa}(t). \tag{9}$$

The SINR requirements ι_u can be different for different vehicles/pedestrians u at short time-scale slot t_τ and vary with time. The following QoS constraints satisfy the achieved SINR which is not greater than the target SINR threshold with the predefined threshold for the probability, ϖ_u:

$$\text{C9: } \Pr\left\{ \sum_{t_\tau \in \mathcal{T}} \sum_{\tilde{n} \in \tilde{\mathcal{N}}} \sum_{q_u \in \mathcal{Q}_u} \sum_{j \in \mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B},\ j\neq u} (\rho_{uj,b}^{\tilde{n}q_u}(t_\tau) + \tilde{\rho}_{uj,b}^{\tilde{n}q_u}(t_\tau)) \right. \left. \tilde{\text{SINR}}_{ij,b}^{\tilde{n}}(t_\tau) \leq \iota_u \right\} \leq \varpi_u. \tag{10}$$

$$\text{C10: } \Pr\left\{ \sum_{t_\tau \in \mathcal{T}} \sum_{n \in \mathcal{N}} \sum_{q_u \in \mathcal{Q}_u} \sum_{j \in \mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B},\ j\neq u} (\rho_{uj,b}^{nq_u}(t_\tau) + \tilde{\rho}_{uj,b}^{nq_u}(t_\tau)) \right. \left. \check{\text{SINR}}_{ij,b}^{n}(t_\tau) \leq \iota_u \right\} \leq \varpi_u. \tag{11}$$

Moreover, the transmit power of node u over all subcarriers assigned to its transmission do not exceed the power budget of node u, $P_u^{\max}$:

$$\text{C11: } \sum_{\kappa \in \mathcal{N}\cup\tilde{\mathcal{N}}} \sum_{q_u \in \mathcal{Q}_u} \sum_{j \in \mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B},\ j\neq u} (\rho_{uj,b}^{\kappa q_u}(t_\tau) + \tilde{\rho}_{uj,b}^{\kappa q_u}(t_\tau)) p_{uj,b}^{\kappa q_u}(t_\tau) \leq P_u^{\max}. \tag{12}$$

Hence, at each short time-scale slot t_τ, the transmission delay for offloading or relaying the qth portion of the task of user u to node $i \in \mathcal{P} \cup \mathcal{V}$, or from node i to node $j \in \mathcal{P} \cup \mathcal{V} \cup \mathcal{R} \cup \mathcal{C}$,

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

$\mathfrak{D}_{ij}^{q_u}(t_\tau)$ is computed as follows:

$$\mathfrak{D}_{ij}^{q_u,\text{Tran}}(t_\tau) = \sum_{\kappa\in\mathcal{N}\cup\tilde{\mathcal{N}}} \frac{[\rho_{ij,b}^{\kappa q_u}(t_\tau) + \tilde{\rho}_{ij,b}^{\kappa q_u}(t_\tau)] z_u \lambda_u^{q_u}}{\mathrm{r}_{ij}^{\kappa}(t_\tau)}. \quad (13)$$

The propagation delay between nodes i and j can be calculated as:

$$\mathfrak{D}_{ij}^{q_u,\text{Prop}}(t_\tau) = \sum_{\kappa\in\mathcal{N}\cup\tilde{\mathcal{N}}} [\rho_{ij,b}^{\kappa q_u}(t_\tau) + \tilde{\rho}_{ij,b}^{\kappa q_u}(t_\tau)] d_{ij}/\nu, \quad (14)$$

where ν is the light speed and d_{ij} is the distance between nodes i and j. In addition, the processing delay for the qth portion of task of user u assigned to node j, i.e., $\mathfrak{D}_j^{q_u,\text{Proc}}(t_\tau)$ can be calculated as follows:

$$\mathfrak{D}_j^{q_u,\text{Proc}}(t_\tau) = \sum_{\kappa\in\mathcal{N}\cup\tilde{\mathcal{N}}} \sum_{\substack{i\in\mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B},\\ i\neq j}} \frac{[\check{\rho}_{j,b}^{q_u}(t_\tau) + \rho_{ij,b}^{\kappa q_u}(t_\tau)] z_u \lambda_u^{q_u} \zeta_o^u}{c_j^{q_u}(t_\tau)}, \quad (15)$$

where $c_j^{q_u}(t_\tau)$ is the CPU cycle allocated to the qth portion of task of user u, respectively. Hence, the task offloading and delivery time for the qth portion of task of user u, i.e., $D_u^{q_u}$ can be calculated as follows:

$$\mathfrak{D}_u^{q_u} = \sum_{t_\tau\in\mathcal{T}} \sum_{i\in\mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B}} \sum_{j\in\mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B},\bar{j}\neq u} [\mathfrak{D}_{ij}^{q_u,\text{Tran}}(t_\tau) + \mathfrak{D}_j^{q_u,\text{Proc}}(t_\tau) + \mathfrak{D}_{ij}^{q_u,\text{Prop}}(t_\tau)]. \quad (16)$$

The total delay of task of user u, $\mathfrak{D}_u$ should not exceed the maximum tolerable delay, $D_u^{\max}$, which are ensured through C12:

$$\text{C12: } \Pr\left\{\mathfrak{D}_u = \sum_{q_u\in\mathcal{Q}_u} \mathfrak{D}_u^{q_u} \leq D_u^{\max}\right\} \geq 1 - \varpi_u. \quad (17)$$

3) Task Migration Model: Due to users' high mobility, they may encounter increased latency during the task offloading process or face task rejection as they move farther away from the corresponding MEC server. Additionally, to balance the load and reduce energy consumption, the network may transfer tasks from the current node to other nodes, such as RSUs, BSs, vehicles, or pedestrians. This enables the network to unload some RSUs or BSs and subsequently power them off, to conserve energy. The task migration plays a vital role in maximizing task completion rates, ensuring QoS for users, reducing energy consumption, and achieving load balancing. For the task migration of the unprocessed part $\bar{q}_u$ of the qth portion of task of user u between two nodes i and j on subcarrier κ at short time-scale slot t_τ, we consider a binary variable as $\mu_{ij}^{\kappa\bar{q}_u}(t_\tau)$, where $\mu_{ij}^{\kappa\bar{q}_u}(t_\tau) = 1$ implies that the unprocessed part $\bar{q}_u$ of task of the qth portion of user u with size of $\bar{q}_u = \lambda_u^{q_u} z_u - [\rho_{ij,b}^{\kappa q_u}(t_\tau - 1) - \mu_{ij}^{\kappa\bar{q}_u}(t_\tau)] c_j^{q_u}(t_\tau)/\zeta_o^u$ bits migrates from node i to node j on subcarrier κ at short time-scale slot t_τ, otherwise $\mu_{ij}^{\kappa\bar{q}_u}(t_\tau) = 0$. In other words, $\mu_{ij}^{\kappa\bar{q}_u}(t_\tau) = 1$ if the unprocessed part $\bar{q}_u$ of the qth portion of task of user u assigned or relayed to node i at short time-scale slot $t_\tau - 1$ is assigned or relayed to node j at short time-scale slot t_τ. Note that there is a relationship between $\rho_{ui,b}^{\kappa q_u}(t_\tau - 1)$ and $\rho_{ij,b}^{\kappa\bar{q}_u}(t_\tau)$, given by $\mu_{ij}^{\kappa\bar{q}_u}(t_\tau) = \rho_{ui,b}^{\kappa q_u}(t_\tau - 1)\rho_{ij,b}^{\kappa\bar{q}_u}(t_\tau)$. The flow conservation constraint also must be satisfied for each migration:

$$\text{C13: } \sum_{\bar{j}} \mu_{i\bar{j}}^{\kappa\bar{q}_u}(t_\tau)\tilde{\rho}_{i\bar{j},b}^{\kappa\bar{q}_u}(t_\tau) - \sum_{\bar{j}} \mu_{\bar{j}i}^{\kappa\bar{q}_u}(t_\tau)\tilde{\rho}_{\bar{j}i,b}^{\kappa\bar{q}_u}(t_\tau) = \begin{cases} 1, & \bar{i} = i, \\ -1, & \bar{i} = j \\ 0, & \text{otherwise.} \end{cases} \quad (18)$$

Migration of the unprocessed part $\bar{q}_u$ of task of the qth portion of user u from its current node to its destination node may change the end-to-end delay of task of user u because the traffic of this service must be re-routed through different paths and the unprocessed part $\bar{q}_u$ must be executed in the new node with different computing capacities. The migration delay from node i to node j for the unprocessed part $\bar{q}_u$ of task of the qth portion of user u at short time-scale slot t_τ can be formulated as:

$$\mathfrak{D}_{ij}^{\bar{q}_u,\text{MG}}(t_\tau) = \mu_{ij}^{\kappa\bar{q}_u}(t_\tau)\left[\frac{\bar{q}_u}{r_{ij,b}^{\kappa}(t_\tau)} + \frac{d_{ij}(t_\tau)}{\vartheta}\right], \quad (19)$$

where $\bar{q}_u/r_{ij,b}^{\kappa}(t_\tau)$ and $d_{ij}(t_\tau)/\nu$ are the transmission and propagation delay for the unprocessed part $\bar{q}_u$ of task of the qth portion of user u. Hence, the additional delay to task of user u due to migration of the unprocessed part $\bar{q}_u$ of task of the qth portion is calculated as follows:

$$\mathfrak{D}_u^{q_u,\text{Add}} = \sum_{t_\tau\in\mathcal{T}} \sum_i \sum_j [\mathfrak{D}_{ij}^{\bar{q}_u,\text{MG}}(t_\tau) + \mathfrak{D}_j^{\bar{q}_u,\text{Proc}}(t_\tau)], \quad (20)$$

where $\mathfrak{D}_j^{\bar{q}_u,\text{Proc}}(t_\tau) = \sum_{t_\tau\in\mathcal{T}}\sum_i \mu_{ij}^{\kappa\bar{q}_u}(t_\tau)\bar{q}_u\zeta_o^u/c_j^{\bar{q}_u}(t_\tau)$ is the processing delay for the unprocessed part $\bar{q}_u$ of task of the qth portion of user u. Note that constrains C1–C14 should be updated based on the migration scheme. For example, in constraint C12 which ensures the total delay of task of user u, $\mathfrak{D}_u$ does not exceed the maximum tolerable delay, $D_u^{\max}$ is updated as follows by adding the additional delay due to migration:

$$\text{C14: } \Pr\{\mathfrak{D}_u \leq D_u^{\max}\} \geq 1 - \varpi_u, \quad (21)$$

where $\mathfrak{D}_u = \sum_{q\in\mathcal{Q}_u}[\mathfrak{D}_u^{q_u} + \mathfrak{D}_u^{q_u,\text{Add}}]$.

D. Demand Uncertainty Modeling

We use an appropriate model to combine Ψ-robust and Γ-robust models. This model is suitable for modeling the stochastic traffic demands because of: 1) Adjusting the number of uncertain task offloading demands by changing Γ; 2) Adjusting the deviation of task size by changing Ψ. In other words, Ψ limits the number of tasks that deviate from their nominal values and Γ limits the maximum deviation magnitude for task size. To model this uncertainty, *box+polyhedral* uncertainty set Λ is defined as $\Lambda = \{\varrho_u | \|\varrho_u\|_\infty \leq \Psi, \|\varrho_u\|_1 \leq \Gamma\}$. It is assumed that the size of tasks fluctuate independently and there is no correlation between task size of different users. Uncertain packet size of each task offloading is denoted by $\tilde{z}_u = \bar{z}_u \pm \varrho_u\hat{z}_u$ where $\tilde{z}_u$ takes a value between $\bar{z}_u - \varrho_u\hat{z}_u$ and $\bar{z}_u + \varrho_u\hat{z}_u$ where $\bar{z}_u$

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

denotes the nominal packet size. The packet size can fluctuate by the positive perturbation $\hat{z}_u$ and ϱ_u.

IV. Problem Formulation and Solution

In this section, the focus is on presenting a scenario based on SDN for task offloading/delivery. The underlying goal is to optimize the network's performance by minimizing both the overall network cost and the task offloading/delivery latency. The SDN-based task offloading/delivery scenario refers to a system where the network's control plane is centralized and programmable, enabling efficient management of tasks and resources. In this context, users can offload their tasks to be processed on other nodes within the network, leading to better resource utilization and performance. We have developed a deterministic optimization problem and enhanced it with a robust scheme to account for uncertainties in traffic demands and channel estimation. The deterministic solution operates based on precise information, while the robust solution considers demand uncertainty and potential variations in the system. This approach allows for more adaptability and flexibility in handling uncertainties and maintaining performance under changing conditions. The goal is to show how the robust solution addresses demand uncertainty compared to the deterministic model. The robust solution algorithm considers demand uncertainty and other potential variations in the system. It takes into account the fact that task offloading demands and network conditions might not be precisely known beforehand. As a result, it adopts a more adaptive and flexible approach to handle uncertainty and maintain acceptable performance even in the face of changing conditions. The aim is to show how the robustification of the solution algorithm overcome the demand uncertainty in the network.

A. Deterministic Formulation of Task Offloading and Delivery Scenario

Here, we maximize successful task offloading (defined as task completion rate) and minimize the total cost due to task offloading, task migration, signaling overhead, etc. Accordingly, our main objective function is as follows:

$$\min_{\substack{\{\tilde{\mathrm{a}}_{br}(\tau)\},\{\check{\rho}^{q_u}_{u,b}(t_\tau)\},\{\rho^{\kappa q_u}_{ui,b}(t_\tau)\},\\ \{\tilde{\rho}^{\kappa q_u}_{ui,b}(t_\tau)\},\{p^{\kappa}_{ij,b}(t_\tau)\},\{\mu^{\kappa q_u}_{ij}(t_\tau)\},\{Q_u\}}} \xi_2\mathfrak{D}-\xi_1\mathfrak{C},$$
$$\text{subject to:} \quad \text{C1–C14}, \tag{22}$$

where, ξ_1 and ξ_2 are two weights for normalizing the impact of network cost $\mathfrak{C}=\sum_\tau\sum_{b\in\mathcal{B}}\phi_b(\tau)+\sum_\tau\sum_{b\in\mathcal{B}}\sum_{r\in\mathcal{R}}\tilde{\mathrm{a}}_{br}(\tau)\tilde{\phi}_{br}(\tau)+\sum_\tau\sum_{b\in\mathcal{B}}\sum_{\bar{b}\in\mathcal{B}/b}\tilde{\phi}^{\text{OverHead}}_{b\bar{b}}(\tau)$ and the task completion rate latency $\mathfrak{D}$ on the objective function. Given that the solution of both optimization problems (i.e., deterministic and robust) are similar (as they differ only in Constraints C4–C6), the solution approach for both of them is the same. Therefore, in the following, we will present the solution method for the optimization problem for robust approach in detail.

B. Robustification of Task Offloading and Delivery Scenario

In this subsection, we introduce a novel robust formulation for uncertain traffic demand because the assumption of deterministic traffic demands is not appropriate in many practical scenarios. In addition, task offloading/delivery by adoptive radio, storage, computing and cost management based on the peak traffic demand is not efficient because the allocated resources are unlikely to be fully utilized most of the time. In addition, if traffic demands fluctuates, it leads to in-feasibility and inability in traffic handling. Therefore, the best way to overcome these challenges and limitations is a robust formulation of our optimization problem under traffic demands uncertainty. To define the robust optimization problem, constraints C4, C5, and C6 should be updated to their robust counterparts as follows for all $\forall i\in\mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B}$:

$$\tilde{\text{C4}}: \Bigg[\sum_{t_\tau\in\mathcal{T}}\sum_{q\in\mathcal{Q}_u}\sum_{\kappa\in\mathcal{N}\cup\tilde{\mathcal{N}}}\sum_{i\in\mathcal{P}\cup\mathcal{V}\cup\mathcal{R},i\neq u}(\check{\rho}^{q_u}_{u,b}(t_\tau)+\rho^{\kappa q_u}_{ui,b}(t_\tau))\lambda^{q_u}_u-1\Bigg]\bar{z}_u+ \max_{\|\varrho_u\|\le\Psi}\Bigg\{\Bigg[\sum_{t_\tau\in\mathcal{T}}\sum_{q\in\mathcal{Q}_u}\sum_{\kappa\in\mathcal{N}\cup\tilde{\mathcal{N}}}\sum_{i\in\mathcal{P}\cup\mathcal{V}\cup\mathcal{R},i\neq u}(\check{\rho}^{q_u}_{u,b}(t_\tau)+\rho^{\kappa q_u}_{ui,b}(t_\tau))\lambda^{q_u}_u-1\Bigg]\varrho_u\hat{z}_u\Bigg\}=0,\forall u,\tau. \tag{23}$$

$$\tilde{\text{C5}}: \sum_{t_\tau\in\mathcal{T}}\sum_{u\in\mathcal{P}\cup\mathcal{V}}\sum_{b\in\mathcal{B}}\sum_{\kappa\in\mathcal{N}\cup\tilde{\mathcal{N}}}\sum_{q_u\in\mathcal{Q}_u}[\check{\rho}^{q_u}_{i,b}(t_\tau)+\rho^{\kappa q_u}_{ui,b}(t_\tau)]\lambda^{q_u}_u\bar{z}_u + \max_{\mathcal{J}\in\mathcal{P}\cup\mathcal{V},|\mathcal{J}|\le\Gamma,\|\varrho_u\|\le\Psi}\Bigg\{\sum_{t_\tau\in\mathcal{T}}\sum_{u\in\mathcal{J}}\sum_{b\in\mathcal{B}}\sum_{\kappa\in\mathcal{N}\cup\tilde{\mathcal{N}}}\sum_{q_u\in\mathcal{Q}_u}[\check{\rho}^{q_u}_{i,b}(t_\tau)+\rho^{\kappa q_u}_{ui,b}(t_\tau)]\lambda^{q_u}_u\varrho_u\hat{z}_u\Bigg\}\le M^{\max}_i, \tag{24}$$

$$\tilde{\text{C6}}: \sum_{t_\tau\in\mathcal{T}}\sum_{u\in\mathcal{P}\cup\mathcal{V}}\sum_{b\in\mathcal{B}}\sum_{\kappa\in\mathcal{N}\cup\tilde{\mathcal{N}}}\sum_{q_u\in\mathcal{Q}_u}[\check{\rho}^{q_u}_{i,b}(t_\tau)+\rho^{\kappa q_u}_{ui,b}(t_\tau)]\lambda^{q_u}_u\bar{z}_u\zeta^u_o + \max_{\mathcal{J}\in\mathcal{P}\cup\mathcal{V},|\mathcal{J}|\le\Gamma,\|\varrho_u\|\le\Psi}\Bigg\{\sum_{t_\tau\in\mathcal{T}}\sum_{u\in\mathcal{J}}\sum_{b\in\mathcal{B}}\sum_{\kappa\in\mathcal{N}\cup\tilde{\mathcal{N}}}\sum_{q_u\in\mathcal{Q}_u}[\check{\rho}^{q_u}_{i,b}(t_\tau)+\rho^{\kappa q_u}_{ui,b}(t_\tau)]\lambda^{q_u}_u\varrho_u\hat{z}_u\zeta^u_o\Bigg\}\le C^{\max}_i, \tag{25}$$

C. The Proposed Two-Time Scale LoMADDPG Framework

We propose an innovative low overhead two-time scale approach using FRL with DDPG agents, termed as LoMADDPG. This novel solution aims to control task offloading and RSU activation/deactivation at two different timescales while ensuring low overhead communication during the training process. The optimization problem (22) is effectively addressed through RL, which proves to be a promising method in dynamically managing task offloading, transmit power, computing and storage RA,

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

as well as RSU activation/deactivation. MDP serves as a classical formalization for sequential decision-making problems, effectively modeling the interaction between the agent and the environment. The utilization of our MDP-based LoMADDPG approach guarantees robustness and adaptability, making it an optimal choice for addressing both robust and deterministic challenges in the system. In our proposed Low Overhead Multi-Agent DDPG (LoMADDPG) algorithm, all agents adjust their actor and critic neural network weights at each time slot under the guidance of the global software-defined networking (SDN) controller (GSC), which serves as a centralized weight tuner.

At the beginning of short time-scale slot t_τ, agent i obtains state $s_i(t_\tau) \in \mathcal{S}$ and then takes action $a_i(t_\tau) \in \mathcal{A}$ according to policy $\pi_i^S(a_i(t_\tau)|s_i(t_\tau))$, where $\mathcal{S}$ and $\mathcal{A}$ denote the sets of states and actions, respectively. At the end of short time-scale slot t_τ, the state $s_i(t_\tau)$ evolves into the next state $s_i(t_\tau+1)$ and the environment returns a reward $r_i(t_\tau) = \mathcal{R}(s_i(t_\tau), a_i(t_\tau), s_i(t_\tau+1))$, where $\mathcal{R}(s_i(t_\tau), a_i(t_\tau), s_i(t_\tau+1))$ is the reward function. The cumulative discounted reward at short time-scale slot t_τ is defined as $R_i(t_\tau+j) = \sum_{j=0}^{\infty} \gamma_j r_i(t_\tau+j)$, where $\gamma_j \in [0,1]$ is the reward discount factor that determines the importance of future rewards. It is worth mentioning that, due to the different temporal characteristics of the task offloading and RSU activation/deactivation frameworks, the allocation of the transmit power, subcarrier assignment, and the decision of RSU activation/deactivation should be updated at different time scales. To deal with the time scale difference between the task offloading and RSU activation/deactivation frameworks, we propose our two-time scale RL framework, where the decision interval for RSU activation/deactivation is extended to the large time-scale slot that includes multiple short time-scale slots. At the beginning of large time-scale slot τ, agent i obtains the updated state $s_i^{\mathrm{L}}(\tau) \in \mathcal{S}_i^{\mathrm{L}}$ and takes action $a_i^{\mathrm{L}}(\tau) \in \mathcal{A}_i^{\mathrm{L}}$ according to its policy $\pi_i^L(a_i^{\mathrm{L}}(\tau)|s_i^{\mathrm{L}}(\tau))$ for the large time-scale control. Based on the current state of the environment, agent i decides the RSU activation/deactivation for large time-scale slot τ. Then, state $s_i^{\mathrm{L}}(\tau)$ evolves into the next state $s_i^{\mathrm{L}}(\tau+1) \in \mathcal{S}_i^{\mathrm{L}}$ and the agent receives reward $r_i^{\mathrm{L}}(\tau) = \mathcal{R}_i^{\mathrm{L}}(s_i^{\mathrm{L}}(\tau), a_i^{\mathrm{L}}(\tau), s_i^{\mathrm{L}}(\tau+1))$ at the end of large time-scale slot τ. Based on the information collected at the large timescale, the policy $\pi_i^L(a_i^{\mathrm{L}}(\tau)|s_i^{\mathrm{L}}(\tau))$ for the large time-scale control can be updated. At the beginning of short time-scale slot t_τ, agent i can obtain state $s_i^{\mathrm{S}}(t_\tau) \in \mathcal{S}_i^{\mathrm{S}}$ and takes action $a_i^{\mathrm{S}}(t_\tau) \in \mathcal{A}_i^{\mathrm{S}}$ according to its policy $\pi_i^S(a_i^{\mathrm{S}}(t_\tau)|s_i^{\mathrm{S}}(t_\tau))$ for the short time-scale control.

Then, state $s_i^{\mathrm{S}}(t_\tau)$ evolves into the next state $s_i^{\mathrm{S}}(t_\tau+1) \in \mathcal{S}_i^{\mathrm{S}}$ and the agent receives reward $r_i^{\mathrm{S}}(t_\tau) = \mathcal{R}_i^{\mathrm{S}}(s_i^{\mathrm{S}}(t_\tau), a_i^{\mathrm{S}}(t_\tau), s_i^{\mathrm{S}}(t_\tau+1))$ at the end of short time-scale slot t_τ. Based on the information collected at the short timescale, the policy $\pi_i^S(a_i^{\mathrm{S}}(t_\tau)|s_i^{\mathrm{S}}(t_\tau))$ for the short time-scale control can be updated. To address two-time scale scheduling, we have created a novel DRL algorithm consisting of two layers, each operating on a distinct time scale. These models allow processes operating at different time scales to effectively cooperate to achieve the system's goals. The key concept behind Layered-DRL is that decisions made at longer timescales influence decisions made at shorter timescales, and conversely, rewards obtained at shorter timescales are used to evaluate decisions made at longer timescales. For RSU activation/deactivation, agent i makes decisions on the large time scale with a decision period Δ. To be specific, a new action a_i^L is made only in decision steps $0, \Delta, 2\Delta, \ldots$. For task offloading/delivery and resource allocation, agent i makes decisions on the short time scale with a decision period δ. To be specific, a new action a_i^S is made only in decision steps $0, \delta, 2\delta, \ldots$; and in other decision steps (i.e., large time scale), the latest action is repeatedly performed. More precisely, at each short time-scale slot, each RSU interacts with the environment to obtain the next state and reward. In addition, at every f short time-scale slots, all RSUs tune their Actor and Critic neural networks' weights to improve their learning process. On the other hand, at each large-scale time slot, each BS interacts with the environment to obtain the next state and reward. At every m large-scale time slot, all BSs tune their Actor and Critic neural networks' weights to improve their learning process.

1) Short Time-Scale RL for Power, Subcarrier, and Task Offloading/delivery Control: In this subsection, we describe our design of RL for the power, subcarrier, and task offloading/delivery control at the short time-scale. The elements of our proposed MDP $\{\mathbf{s}_i^{\mathrm{S}}(t_\tau), \mathbf{a}_i^{\mathrm{S}}(t_\tau), \mathbf{r}_i^{\mathrm{S}}(t_\tau)\}$ are defined as the state, action, reward function at short time-scale slot t_τ at agent i, respectively.

- *State* $\mathbf{s}_i^{\mathrm{S}}(t_\tau)$: At each short time-scale slot t_τ, the state for agent i is given as $\mathbf{s}_i^{\mathrm{S}}(t_\tau) = \{\boldsymbol{\varphi}_i(t_\tau), \boldsymbol{O}_i(t_\tau), \mathbf{C}_i^{\mathrm{rem}}(t_\tau), \mathbf{M}_i^{\mathrm{rem}}(t_\tau), \mathbf{SINR}_i(t_\tau)\} \in \mathcal{S}_i^{\mathrm{S}}$, where $\boldsymbol{\varphi}_i(t_\tau) = [\varphi_{v_1}(t_\tau), \varphi_{v_2}(t_\tau), \ldots, \varphi_{v_V}(t_\tau), \varphi_{p_1}(t_\tau), \varphi_{p_2}(t_\tau), \ldots, \varphi_{p_P}(t_\tau)]$ denotes the position of users (i.e., vehicles and pedestrians) for corresponding agent i. $\mathcal{O}_i(t_\tau)$, $\mathbf{C}_i^{\mathrm{rem}}(t_\tau)$, and $\mathbf{M}_i^{\mathrm{rem}}(t_\tau)$ are the feature of users' task (defined before) and the residual computation and storage resources of all MEC-enabled devices of the corresponding agent, respectively. $\mathbf{SINR}_i(t_\tau) = [\mathrm{SINR}_{i1,1}^{\mathrm{I}}(t_\tau), \ldots, \mathrm{SINR}_{ij,b}^{\kappa}(t_\tau), \ldots, \mathrm{SINR}_{iJ,B}^{N+\tilde{N}}(t_\tau)]$ denotes the SINR (in dB) of vehicles/pedestrians at the beginning of short time-scale slot t_τ.
- *Action* $\mathbf{a}_i^{\mathrm{S}}(t_\tau)$: The action at each short time-scale t_τ includes $\mathbf{a}_i^{\mathrm{S}}(t_\tau) = \{\{\tilde{\rho}_{u,b}^{q_u}(t_\tau)\}, \{\rho_{ui,b}^{\kappa q_u}(t_\tau)\}, \{\tilde{\rho}_{ui,b}^{\kappa q_u}(t_\tau)\}, \{p_{ij,b}^{\kappa}(t_\tau)\}, \{\mu_{ij}^{\kappa \tilde{q}_u}(t_\tau)\}, \{Q_u\}\} \in \mathcal{A}_i^{\mathrm{S}}$.
- *Reward* $r_i^{\mathrm{S}}(t_\tau)$: The objective of controlling the transmit power and subcarrier assignment is to guarantee that the SINR of each vehicle/pedestrian achieves the target threshold with a high probability as in (10) and (11). In addition, the objective of controlling transmit power, subcarrier assignment, task offloading/delivery, and migration decisions is to the delay of each vehicle/pedestrian achieves the target threshold with a high probability as in (21). As the transmit power control, subcarrier assignment, task offloading/delivery, and migration decisions do not have direct impact on system cost and there is no RSU activation/deactivation at the short time-scale, the objective of transmit power control, subcarrier assignment, task offloading/delivery, and migration decisions can be reduced to increase the instantaneous SINR of vehicles/pedestrians and reduce the instantaneous delay of

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

vehicles/pedestrians given the current active RSUs. Thus, the reward of our proposed solution at short time-scale slot t_τ is designed as $r_i^{\mathrm{S}}(t_\tau) = \min_u\{\mathrm{SINR}_u(t_\tau) - \iota_u\} + \min_u\{\mathfrak{D}_u - D_u^{\max}\}$, where

$$\mathrm{SINR}_u(t_\tau) = \begin{cases} \sum_{\tilde{n}\in\tilde{\mathcal{N}}}\sum_{q_u\in\mathcal{Q}_u}\sum_{j\in\mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B},j\neq u}(\rho_{uj,b}^{\tilde{n}q_u}(t_\tau)+\tilde{\rho}_{uj,b}^{\tilde{n}q_u}(t_\tau)) \\ \cdot\tilde{\mathrm{SINR}}_{uj,b}^{\tilde{n}}(t_\tau), \quad \text{If } \kappa=\tilde{n}, \\ \sum_{n\in\mathcal{N}}\sum_{q_u\in\mathcal{Q}_u}\sum_{j\in\mathcal{P}\cup\mathcal{V}\cup\mathcal{R}\cup\mathcal{B},j\neq u}(\rho_{uj,b}^{nq_u}(t_\tau)+\tilde{\rho}_{uj,b}^{nlq_u}(t_\tau)) \\ \cdot\check{\mathrm{SINR}}_{uj,b}^{n}(t_\tau), \quad \text{If } \kappa=n. \end{cases} \tag{26}$$

2) Large Time-Scale RL for RSU Activation/deactivation Control: The agent (i.e., each BS) controls the corresponding RSUs activation/deactivation at the large time-scale slots to minimize the system cost. In this Subsection, we describe our designs of RL for the RSUs activation/deactivation at the large time-scale. The elements of our proposed MDP $\{\mathbf{s}_i^{\mathrm{L}}(\tau), \mathbf{a}_i^{\mathrm{L}}(\tau), \mathbf{r}_i^{\mathrm{L}}(\tau)\}$ are defined as the state, action, and reward function at short time-scale slot τ at agent i, respectively.

- *State* $\mathbf{s}_i^{\mathrm{L}}(\tau)$: The algorithm for transmit power control, subcarrier assignment, task offloading/delivery, and migration decisions in the previous subsection only focuses on the improvement of communication performance and task processing delay, i.e., achieving high SINR values and low task processing delay for all vehicles/pedestrians. However, with regard to the system cost minimization, the agents (i.e., BSs) need to make decisions on RSUs activation/deactivation at the large time-scale slots while balancing the trade-off between system cost and communication and task processing efficiency. Thus, the information reflecting communications, task processing, and vehicles/pedestrians dynamics is necessary to decide the RSUs activation/deactivation. In addition to the designed state for the transmit power control, subcarrier assignment, task offloading/delivery, and migration decisions, the vehicles/pedestrians dynamics and RSUs state are added into the state space for controlling the RSUs. Therefore, at each large time-scale slot τ, the state for agent i is given as $\mathbf{s}_i^{\mathrm{L}}(\tau) = \{\boldsymbol{\varphi}_i'(\tau), \boldsymbol{O}_i'(\tau), \mathbf{C}_i^{\mathrm{rem}'}(\tau), \mathbf{M}_i^{\mathrm{rem}'}(\tau), \mathbf{SINR}_i'(\tau), \boldsymbol{\phi}(\tau), \tilde{\boldsymbol{\phi}}(\tau)\} \in \mathcal{S}_i^{\mathrm{L}}$. The first five entries of state $\mathbf{s}_i^{\mathrm{L}}(\tau)$ are the same as the entries of state $\mathbf{s}_i^{\mathrm{S}}(\tau)$ except the difference in time-scale. The estimated normalized $\boldsymbol{\varphi}_i'(\tau)$ of position of users, $\boldsymbol{O}_i'(\tau)$ of feature of user' task, residual computation resource $\mathbf{C}_i^{\mathrm{rem}'}(\tau)$, residual storage resource $\mathbf{M}_i^{\mathrm{rem}'}(\tau)$, and $\mathbf{SINR}_i'(\tau)$ can be expressed as $\boldsymbol{\varphi}_i'(\tau) = \boldsymbol{\varphi}_i(\tau T)$, $\boldsymbol{O}_i'(\tau) = \boldsymbol{O}_i(\tau T)$, $\mathbf{C}_i^{\mathrm{rem}'}(\tau) = \mathbf{C}_i^{\mathrm{rem}}(\tau T)$, $\mathbf{M}_i^{\mathrm{rem}'}(\tau) = \mathbf{M}_i^{\mathrm{rem}}(\tau T)$, $\mathbf{SINR}_i'(\tau) = \mathbf{SINR}_i(\tau T)$, respectively. Other entries in the state tuple are $\boldsymbol{\phi}(\tau) = [\phi_1(\tau), \ldots, \phi_b(\tau), \ldots, \phi_B(\tau)]$, $\tilde{\boldsymbol{\phi}}(\tau) = [\tilde{\phi}_{11}(\tau), \ldots, \tilde{\phi}_{br}(\tau), \ldots, \tilde{\phi}_{BR}(\tau)]$ at the beginning of large time-scale slot τ.
- *Action* $a_i^{\mathrm{L}}(\tau)$: The action at each large time-scale τ includes $a_i^{\mathrm{L}}(\tau) = \{\tilde{\mathbf{a}}_b(\tau)\} \in \mathcal{A}_i^{\mathrm{L}}$, where $\tilde{\mathbf{a}}_b(\tau) = \{\tilde{a}_{b1}(\tau), \tilde{a}_{b2}(\tau), \ldots, \tilde{a}_{bR}(\tau)\}$ is a vector that defines the active and standby RSUs.
- *Reward* $r_i^{\mathrm{L}}(\tau)$: The objective of RSUs activation/deactivation is to minimize the system cost while guaranteeing the communication QoS and satisfying task processing delay. Thus, under the premise that the communication QoS and satisfying task processing delay constraints are satisfied, the agent should receive a positive reward for achieving a lower system cost. If the communication QoS and satisfying task processing delay constraints are not met, then the agent should be punished. Therefore, the reward $r_i^{\mathrm{L}}(\tau)$ for RSUs activation/deactivation is designed as follows:

$$r_i^{\mathrm{L}}(\tau) = \begin{cases} \Omega(D - (\xi_2\bar{\mathfrak{D}} - \xi_1\mathfrak{C})), \\ \max_u \chi_u(\tau) \le \varpi \ \& \ \max_u \tilde{\chi}_u(\tau) \ge 1-\varpi, \\ -\max_u \chi_u(\tau) - \max_u \tilde{\chi}_u(\tau), \\ \max_u \chi_u(\tau) > \varpi \ \& \ \max_u \tilde{\chi}_u(\tau) < 1-\varpi, \end{cases} \tag{27}$$

where $\bar{\mathfrak{D}} = \frac{1}{VP\Delta}\sum_{u\in\mathcal{U}}\int_{\tau\Delta}^{(1+\tau)\Delta}\mathfrak{D}_u(t_\tau)dt_\tau$ denotes the average delay, Ω is the reward scaling coefficient and $\chi_u = \tilde{\Pr}\{\mathrm{SINR}_u(t_\tau) \le \iota_u\}, \tau T \le t_\tau < (\tau+1)T$ denotes the estimated probability that the target SINR is not achieved for vehicle/pedestrian u during large time-scale slot τ. The probability is computed as $\tilde{\Pr}\{\mathrm{SINR}_u(t_\tau) \le \iota_u\} = \frac{1}{T}\sum_{t_\tau=\tau T}^{(1+\tau)T}\mathbb{I}\{\mathrm{SINR}_u(t_\tau) \le \iota_u\}$, and $\tilde{\chi}_u = \tilde{\Pr}\{\mathfrak{D}_u \le D_u^{\max}\}, \tau T \le t_\tau < (\tau+1)T$ denotes the probability that the target delay is not achieved for vehicle/pedestrian u during large time-scale slot τ. This probability is computed as $\tilde{\Pr}\{\mathfrak{D}_u \le D_u^{\max}\} = \frac{1}{T}\sum_{t_\tau=\tau T}^{(1+\tau)T}\mathbb{I}\{\mathfrak{D}_u \le D_u^{\max}\}$. $D_u^{\max}$ is a predefined constant to ensure that reward $r_i^{\mathrm{L}}(\tau)$ is positive when communication QoS and task processing delay constraints are satisfied. Our approach utilizes a multi-agent Actor-Critic framework in a mixed cooperative-competitive setting, addressing challenges faced by conventional algorithms in scenarios with multiple agents. By adapting actor-critical techniques to account for the action policies of other agents, we can effectively learn intricate coordination policies. Our FRL algorithm, based on DDPG, focuses on Low overhead Multi-Agent DDPG with the support of a GSC as a global weight tuner. The proposed LoMADDPG solution involves all agents adjusting their neural network weights with the assistance of the GSC. Additionally, we have implemented a prioritization system to enhance system performance. Inspired from FRL method, the architecture of our proposed LoMADDPG is depicted in Fig. 3, where the agents (RSUs or BSs) have different interactions. The order of the interactions is depicted with the yellow triangles (considering our proposed two-time scale in Fig. 2). More precisely, at each short time-scale slot t_τ each RSU interacts with the environment to obtain the next state and reward. In addition, at every f short time-scale slot t_τ, i.e., ft_τ, all RSUs tune their Actor and Critic neural networks' weights to improve their learning process. On the other hand, at each large-scale time slot τ each BS interacts with the environment to obtain next state and reward. At every

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

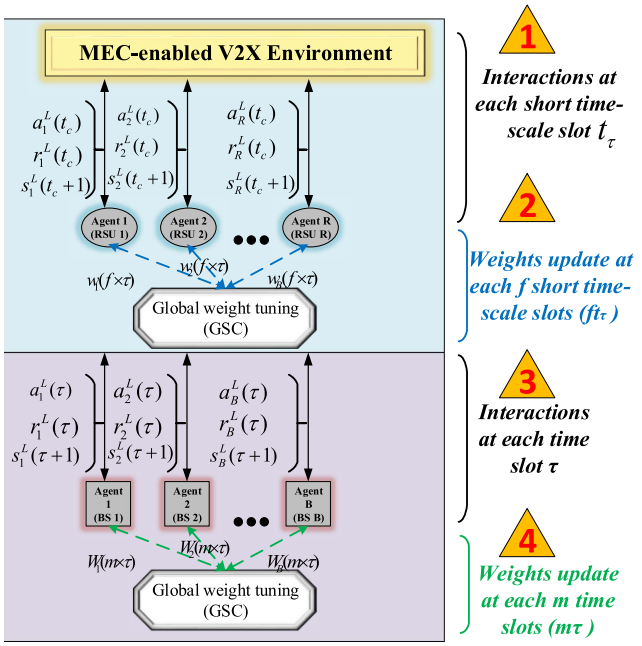


Fig. 3. Architecture of our proposed LoMADDPG solution framework.

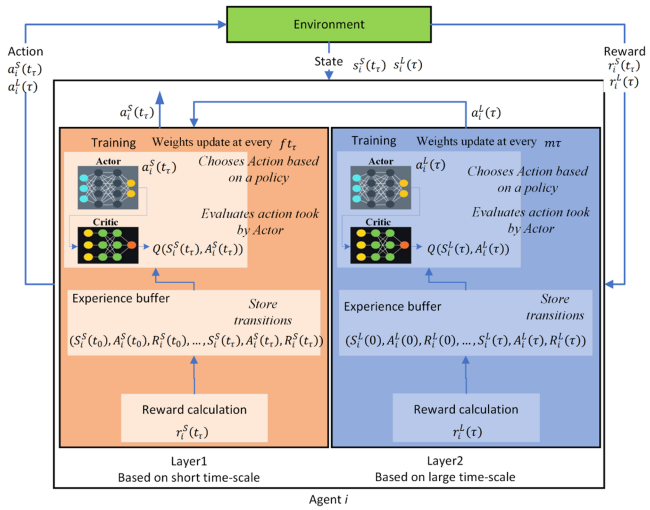


Fig. 4. The architecture of two-time scale agent.

m large-scale time slots τ, i.e., $m\tau$, all BSs tune their Actor and Critic neural networks' weights to improve their learning process. It is worth noting that for more clearance we draw two GSCs in Fig. 3, but keep in mind that there is only one GSC in our system. Fig. 4 demonstrates inside of each RSU or BS as agent.

Please note that the decentralized nature of our system model, in which the agents' interactions are identical, results the agents to trap in local optimum. Hence, to achieve global optimum in our system model, the agents need to have communication with each other. On the other hand, the communication between agents poses a huge amount of overhead, specially when the number of agents increases. Therefore, we propose our LoMADDPG solution algorithm based on the federated learning proposed in [27], so that at every specific period, all agents tune their Actor and Critic neural network weights (W) by the help of GSC which acts as global wight tuner. To enhance the performance of the whole system, we prioritized the agents so that the weights of the agents with higher priorities have higher impacts in tuning procedure. Accordingly, the weights of each agents' neural networks are tuned as follows:

$$W_i(t_\tau T + t_\tau) = W_i(t_\tau T)\frac{\sum_{i\in I}\phi_i W_i}{I}, \tag{28}$$

where ϕ_i is the priority of the ith agent which is calculated as follows:

$$\phi_i = \frac{\sum_{t=0}^{t_\tau T} R_i(t_\tau)}{\sum_{t=0}^{t_\tau T}\sum_{i\in I} R_i(t_\tau)}. \tag{29}$$

It is worth noting that the weight tuning procedure happens at both short-scale and large-scale time slots for RSUs and BSs, respectively. The neural network weight tuning for the agent improves the agents' communication efficiency by decreasing the amount of data that needs to be shared. In addition, since the agents do not share their states and action, i.e., their transitions, our proposed method (the same as federated learning) preserves the privacy of the agents. Algorithm 1 depicts the whole procedure of our proposed LoMADDPG solution algorithm. Note that at each $\mathfrak{E}_R$ short-scale time slot, all RSUs locally update their Actor and Critic neural networks (for local training). This procedure is done in Lines 15–19. On the other hand, all RSUs tune their Actor and Critic neural network weights at each ft_τ short-scale time slots (for global learning). This procedure is done in Lines 20–22. The same procedure is done for the BS; in Lines 28–32 each BS updates its Actor and Critic neural network weights at each episode $\mathfrak{E}_B$, and all BSs perform global update in Line 33–35 at each $m\tau$ large-scale time slots.

D. Computational Complexity

In this section, we determine the computational complexity of our proposed algorithm which consists of action selection and the training process complexity.

1) Computational Complexity of Action Selection: The computational complexity for action selection will depend on the algorithm used for coordination and decision-making. We assuming that each agent independently selects an action based on its current state, meaning that each agent collects data and performs local computations to select an action based on its state S. Hence, the time complexity for action selection for each agent is $\mathcal{O}(S)$. We assume that each agent is a fully connected neural network with fixed number of M hidden layers and fixed number of N neurons in each hidden layer. The computational complexity of action calculation involves summing over the products of size of two consecutive layers, taking into account the operations related to activation functions. i.e., total number of operations required to perform a feedforward pass through the fully connected neural network [29]. Therefore, the computational complexity of action selection for our proposed LoMADDPG for each agent is $\mathcal{O}(S\times N + N + N^2 + N\times A + A) = \mathcal{O}(N^2)$. Uniform power distribution over subcarriers simplifies the optimization problem by reducing the dimensionality of the action space,

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: The Two-Time-Scale LoMADDPG Solution Framework.

Input: Initialize the neural network parameters of all agents' Actors and Critics networks, $\mathbf{W}_{\mu}(\mathbf{0})$ and $\mathbf{W}_{\mathbf{\Theta}}(\mathbf{0})$ with random values.
Input: Initialize the neural network parameters of all agents' target Actors and target Critics networks, $\mathbf{W}'_{\mu}(\mathbf{0})$ and $\mathbf{W}'_{\mathbf{\Theta}}(\mathbf{0})$ with random values.
Input: Initialize the replay buffer for all agents, i.e., D_i for i^{th} agent.
Input: Set the neural network update frequency $\mathfrak{E}_R$ and $\mathfrak{E}_B$ for short time scale and large time scale, respectively..
♯ **Large Time Scale Iteration (for BSs)**:
for $\tau = 1 : m\tau$ **do**
  Each BS b receives state $s_b^L(\tau)$ from the environment.
  Each BS b selects $a_b^L(\tau)$ (which RSUs should be turned off) and executes it.
  ♯ **Short Time Scale Iteration (for RSUs)**:
  for $t_\tau = 1 : ft_\tau$ **do**
    Each RSU r receives state $s_r^S(t_\tau)$ from the environment.
    Each RSU r selects $a_r^S(t_\tau)$ (Subcarrier assignment, task offloading etc.) and executes it.
    Each RSU r receives its reward $r_r^S(t_\tau)$.
    Each RSU r receives the new state $s_r^S(t_\tau + 1)$ from the environment.
    Each RSU r stores the transition $(s_r^S(t_\tau), a_r^S(t_\tau), r_r^S(t_\tau), s_r^S(t_\tau + 1))$ into its experience replay D_r.
    ♯ **RSUs local training:**
    if $t_\tau \mod \mathfrak{E}_R == 0$ **then**
      Each RSU r randomly samples a mini-batch from its replay memory D_r.
      Each RSU r update the weights of its Actor and Critic networks using Adam optimizer [28].
      Each RSU updates the weights of its target Actor and target Critic networks by $W'_{\mu_r} \leftarrow \mathfrak{E}_R W_{\mu_r}$ $+(1 - \mathfrak{E}_R)W'_{\mu_r}$, and $W'_{\theta_r} \leftarrow \mathfrak{E}_R W_{\theta_r} + (1 - \mathfrak{E}_R)W'_{\theta_r}$.
    end if
    ♯ **RSUs weight aggregation process (RSUs federation):**
    if $t_\tau \mod f == 0$ **then**
      All RSUs tune their Actor and Critics weights by (28).
    end if
  end for
  Each BS b receives its reward $r_b^L(\tau)$.
  Each BS b receives the new state $S_b^L(\tau + 1)$ from the environment.
  Each BS b stores the transition $(s_b^L(\tau), a_b^L(t), r_b^L(\tau), s_b^L(\tau + 1)$ into its experience replay D_b.
end for
♯ **BSs local training:**
if $\tau \mod \mathfrak{E}_B == 0$ **then then**
  Each BS b randomly samples a mini-batch from the replay memory D_b.
  Each BS b updates the weights of its Actor and Critic networks using Adam optimizer [28].
  Each BS b updates the wights of the target Actor and target Critic networks by $W'_{\mu_b} \leftarrow \mathfrak{E}_B W_{\mu_b} + (1 - \mathfrak{E}_B)W'_{\mu_b}$, and $W'_{\theta_b} \leftarrow \mathfrak{E}_B W_{\theta_b} + (1 - \mathfrak{E}_B)W'_{\theta_b}$.
end if
♯ **BSs weight aggregation process (BSs federation):**
if $\tau \mod m == 0$ **then then**
  All BSs tune their Actor and Critic networks weights by (28).
end if

effectively removing the power allocation variable. Therefore, the action space dimension is reduced to $\mathcal{A}' = \mathcal{A} \setminus \{p_{ij,b}^{\kappa}(t_\tau)\}$, were $|\mathcal{A}'| < |\mathcal{A}|$.

2) Computational Complexity of Training: The computational complexity of the training process of the neural network is equal to the total number of neural network parameters (weights and biases). Since our proposed LoMADDPG uses DDPG for each agent, we break down the computational complexity into actor network training and critic network training. The training of the actor network involves forward passes to predict actions and backward passes for gradient updates. The time complexity for the actor network training (per agent, per iteration) is $\mathcal{O}(T \times N \times S)$. Similarly, the time complexity for the critic network training (per agent, per iteration) is $\mathcal{O}(T \times N \times S)$.

3) Communication Overhead Complexity: The complexity for sending actor/critic weights from the central coordinator to each agent (per iteration) is $\mathcal{O}(S \times N + N^2 + N \times A)$. Therefore, the total computational complexity for the training process in the federated learning with N agents using the DDPG method, including the batch size B and communication overhead, where each agent's actor and critic networks have M hidden layers with N neurons, is $\mathcal{O}(T \times B(S \times N + N^2 + N \times A))$.

V. Performance Assessment

A. Simulation Setup

In this section, we assess the performance of our proposed LoMADDPG algorithm based on the real traffic data. We compare our proposed system model (with demand uncertainty and without demand uncertainty) and our proposed solution algorithm with the following methods and algorithms:

- *No-Partition:* The method in which for system model without demand uncertainty, there is no task partitioning for task offloading. For the solution algorithm, we consier our proposed LoMADDPG method.
- *No-Migration:* The method in which for system model without demand uncertainty, there is no task migration for task offloading. For the solution algorithm, we consier our proposed LoMADDPG method.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SIMULATION PARAMETERS [30], [31]

Parameter	Value
The number of BSs	3
The number of RSUs controlled by each BS	5
Computational capability of each Vehicle	5–10 MHz
Caching (storage) capacity of each Vehicle	3 tasks (max 300 MB)
Computational capability of each Pedestrian	4–8 MHz
Caching (storage) capacity of each Pedestrian	2 tasks (max 200 MB)
Computational capability of each RSU	9–21 GHz
Caching (storage) capacity of each RSU	8 tasks (max 800 MB)
Computational capability of each BS	30 GHz
Caching (storage) capacity of each BS	10 tasks (max 1 GB)
Bandwidth of C-V2X	100 MHz
Bandwidth of DSRC	20 MHz
Task nominal packet size	1–150 MB
Uncertainty of demands	5–45 %
Maximum number of partitions	10
Task latency constraint	200–600 ms
Learning rate of critics network for MADDPG and LoMADDPG	0.001
Discount factor for MADDPG and LoMADDPG	0.95
Target network update frequency for MADDPG and LoMADDPG	1000
Number of hidden layers for each agent in MADDPG and LoMADDPG	2
Number of neurons for each layer for MADDPG and LoMADDPG	512
Replay buffer size for MADDPG and LoMADDPG	10 000

- *MADDPG:* In this benchmark, we consider MADDPG as a solution to our proposed problem when there is no demand uncertainty. We consider this benchmark in order to demonstrate the advantage of our proposed LoMADDPG solution.

We aim to provide a thorough and comprehensive comparison of the proposed algorithm with the state-of-the-art methods. The primary objective of our work is to address the limitations observed in the existing algorithms, such as the No-partition and No-migration approaches. These algorithms were selected intentionally to serve as representative baseline methods, as they have been widely used in previous research and provide a foundation for comparison. The detailed experimental parameters are listed in Table I. [2]

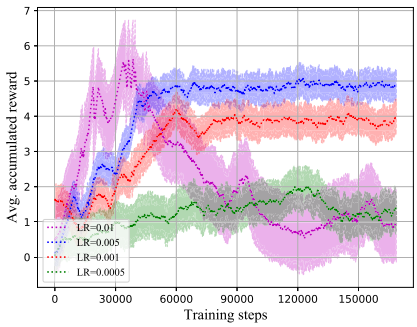
B. Simulation Results

We discuss the performance of our novel LoMADDPG and existing MADDPG by comparing our proposed model with the MADDPG. We also evaluate the smart dynamic task partitioning and cooperative offloading/delivery mechanism (which are proposed for our system model) by comparing it with the two No-Partition and No-Migration baseline approaches. Moreover, we demonstrate the performance gain of considering demand uncertainty which provide robustness to our model by comparing our system model with and without uncertainty. To do so, we assume that there is demand uncertainty in the environment, therefore the agents are able to learn the uncertainty. We will discuss more about this later in the results. Please note that we name our proposed LoMADDPG method trained with demand uncertainty as "The robust scheme". Moreoever, we name our proposed LoMADDPG method, trained without demand uncertainty as "The unrobust scheme". Also, after training, all methods are tested in a system model which has demand uncertainty. Hence, better performance of "The robust scheme" compared to "The unrobust scheme" is expected. We initially set the number of partitions for each task to be 5. Later, we will change the number of partitions for more analysis. We consider the cost and task completion rate as two criteria for the comparison. The cost refers to sum of all costs due to bandwidth consumption in Downlink (DL) and Uplink (UL), bandwidth consumption in backhaul communication between agents, task processing cost, as well as task migration cost. We consider a task as completed when that task is successfully offloaded, processed, and delivered back to the corresponding user u within a time duration less than it's maximum tolerable delay $D_u^{\max}$, regardless of how many task migration and task partitioning was involved for this task. For instance, if at least one of the partitions of a task fails during offloading process (it can happen during UL transmission, during processing by MEC server, or during DL delivery), the whole task is considered as not completed task (failed).

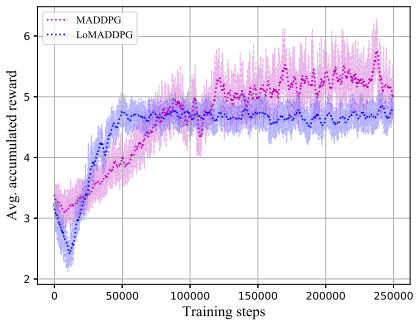
We first show the convergence analysis of our proposed LoMADDPG for different learning rates of agents' actor network in Fig. 5(a). As depicted, the best performance is obtained when the learning rate of all agents' actor network is equal to 0.005. The shaded light colors are the actual reward which shows the fluctuations during training. For more clearance, we also plot the moving average lines inside the shaded curves for each method. Please note that the simulation is performed with a random seed, which means every run may have different results from each other. However, the trend is always the same and the point of convergence is approximately the same for different runs. Hence, we have plotted one of the runs in these figures [32], [33]. It is also worth noting that the learning rate of critic networks is fixed and is set to 0.001 for both MADDPG and LoMADDPG. To compare the convergence of MADDPG and LoMADDPG algorithms, the accumulated reward is shown in Fig. 5(b). As demonstrated, our proposed low overhead LoMADDPG converges faster than MADDPG method. However, MADDPG achieves higher reward at the cost of communication overhead between agents which results in more reward (we will analyze the overhead later in this section). The convergence speed of our algorithm is relatively slower than the state-of-the-art methods, such as MADDPG, due to the inherent nature of federated learning based on which our proposed LoMADDPG operates [32], [33]. Federated learning involves training agents locally on their respective data, which can lead to slower convergence when compared to centralized methods like MADDPG. However, the advantage of federated

[2]The source code is available at https://dx.doi.org/10.21227/pemn-g388.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

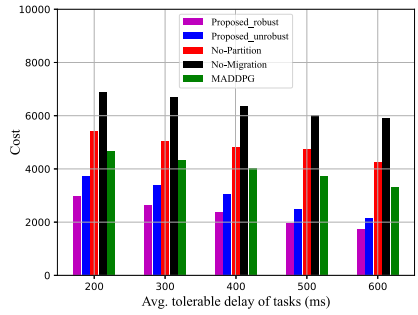


(a)

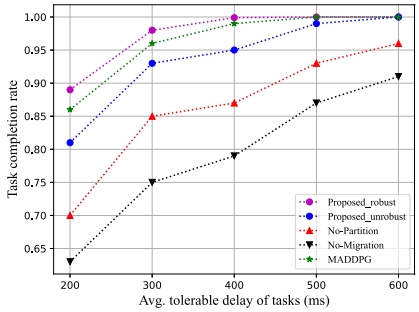


(b)

Fig. 5. Average accumulated reward for proposed LoMADDPG and MADDPG.



(a)



(b)

Fig. 6. The impact of increasing the tolerable delay on (a) cost and (b) task completion rate.

learning lies in its ability to prioritize privacy preservation and mitigate communication overhead. By allowing agents to train on their local data without the need to centrally aggregate data, federated learning inherently incorporates privacy-preserving mechanisms. This decentralized approach significantly reduces the amount of communication required during training, which is especially beneficial in scenarios where data sharing is restricted or privacy concerns are paramount. It's important to note that the trade-offs between convergence speed, privacy preservation, and communication overhead are central to the design of our algorithm. We believe that in scenarios where privacy and limited communication are critical, our algorithm's slower convergence can be outweighed by its distinct advantages in maintaining data privacy and minimizing communication costs.

In the remainder of this section, we show the cost and task completion rate comparison for all methods versus average delay, number of users, and number of partitions. The demand uncertainty is considered as 25%. We later analyze the impact of demand uncertainty by increasing it from 5% to 45%.

Fig. 6(a) depicts the cost versus the average tolerable delay of the tasks. All methods are tested with demand uncertainty equal to 25%. However, only the robust scheme is trained in a system with 25% demand uncertainty. Others are trained in a system with no uncertainty. The number of users is 70 and the number of partitions is 5. The tolerable delay of each user's task is a duration in which the task must be offload, processed, and delivered to the user, apart from how many partitions is the task decomposed to. For instance, let 300 ms be the tolerable delay for a particular user's task. Then if this task is partitioned into five different parts (no matter if the size of each part is different) and each part has been offloaded to different MEC servers, then all of the results of processing of the task must be delivered to the users before 300 ms. In this case we declare the task as completed, which accounts for the completion rate. Expectedly and as demonstrated in Fig. 6(a), increasing the average tolerable delay of tasks causes the RSUs to perform a task offloading scheme that imposes a lower cost to the system as there would be higher delay margin for offloading which lets the users to hold their tasks to be offloaded on low cost MEC devices. Comparing our proposed method with No-Partition method, one can observe that our proposed task partitioning decreases the cost by 42%. In addition, compared to No-Migration method, our proposed method with task migration decreases the cost by 76%. Moreover, when our proposed LoMADDPG trained in the environment with task demand uncertainty, it outperforms the one which is not trained in the uncertain environment Comparing our Unrobust scheme with MADDPG, we can observe that using MADDPG as a solution for our problem increases the cost of task offloading by 45%. This is because agents' communication in the MADDPG method is higher compared to the LoMADDPG, which drastically increases the cost.

As far as the task completion rate is concerned, expectedly and as in Fig. 6(b), the task completion rate increases as the average tolerable delay of tasks increases since the tasks can tolerate more delay and the MEC devices have more capacity to process the offloaded tasks. We can see that for delay of 300 ms, the task migration and partitioning can increase the task completion rate by about 10% and 20%, respectively. It is worth noting that considering MADDPG as solution, the task completion rate

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

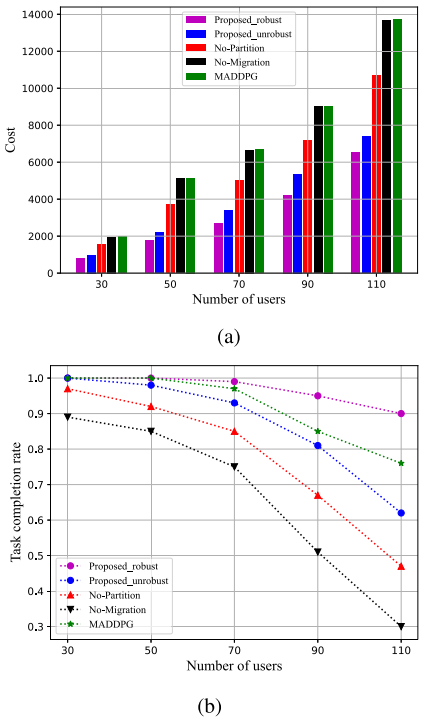


Fig. 7. The cost and task completion rate versus the number of users.

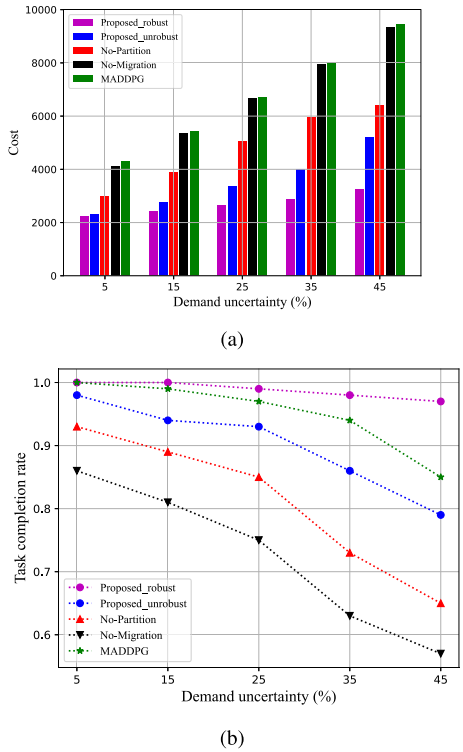


Fig. 8. The cost and task completion rate versus the demand uncertainty.

increases by about 5% compared to our unrobsut LoMADDPG, but this is at the expense of communication overhead between agents (RSUs and BSs) and GSC. At the same time, our robust solution all methods for all values of delay.

The impact of the number of users on the cost and task completion rate are shown in Fig. 7(a) and (b), respectively. All methods are tested with demand uncertainty equal to 25%. However, only the robust scheme is trained in a system with 25% demand uncertainty. Others are trained in a system with no uncertainty. The tolerable delay for all tasks is 300 ms and the number of partitions is 5. As can be seen, even with the low number of users (i.e., 30 users), the method without the task partitioning (No-Partition) and the migration scheme (No-Migration) cannot achieve 100% task completion rate. This indicates the importance of task migration and partitioning in the offloading mechanism. It is worth noting again that although MADDPG (MADDPG) achieves better task completion rate compared to the unrobust scheme, its task offloading cost is much higher due to agents' communication overhead. Simialr to the previous figure, our robust scheme outperforms all other scenarios in both cost and performance.

In order to show the superiority of our proposed demand uncertainty model, we compare it with demand deterministic (the unrobust scheme) method by increasing the level of uncertainty from 5% to 45%. All methods are tested in a system with demand uncertainty from 5% to 45%. However, only the robust scheme is trained in a system with demand uncertainty. Others are trained in a system with no uncertainty. The number of users is 70, the tolerable delay of all tasks is 300 ms, and the number of partitions is 5. As demonstrated in Fig. 8, when the demand uncertainty increases, the performance gap between our proposed method with and without uncertainty increases. To show the impact of task partitioning, we analyze our proposed method for different maximum number of task partitions. We let the MEC devices to offload their tasks to up to 5, 8, 11, 14, and 17 partitions and analyze the cost and task completion rate. Fig. 9 demonstrates the impact of number of partitions on the cost and task completion rate. All methods are tested with demand uncertainty equal to 25%. However, only the robust scheme is trained in a system with 25% demand uncertainty. Others are trained in a system with no uncertainty. The number of users is 70 and the tolerable delay of all tasks is 300 ms. We can observe that by increasing the number of partitions, the cost is decreased. However, for the number of partitions greater that 11, the decrease in cost is negligible. Also, for the task completion rate, one can conclude that the increase of partitions causes the task completion rate to increase such that for the number of partitions equal to 17, all methods achieved 100% task completion rate.

We now compare our LoMADDPG solution with MADDPG from overhead perspective. In our LoMADDPG solution, the agents update their weights at specific periods (RSUs at each ft_τ short-scale time slots and BSs at each $m\tau$ large-scale time slots) by sending and receiving their neural network weights to the GSC (as global server for weight tuning). In MADDPG, each agent sends its actions combined with its local state to the GSC (as a centralized critic) for centralized training. As such, a huge amount of data is transmitted and received (between RSUs and GSC at each short-scale time slot t_τ and between BSs and GSC at each large-scale time slot τ). On the other hand, the state space in the considered environment consists of values representing the task data rate and task tolerable delay typically

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

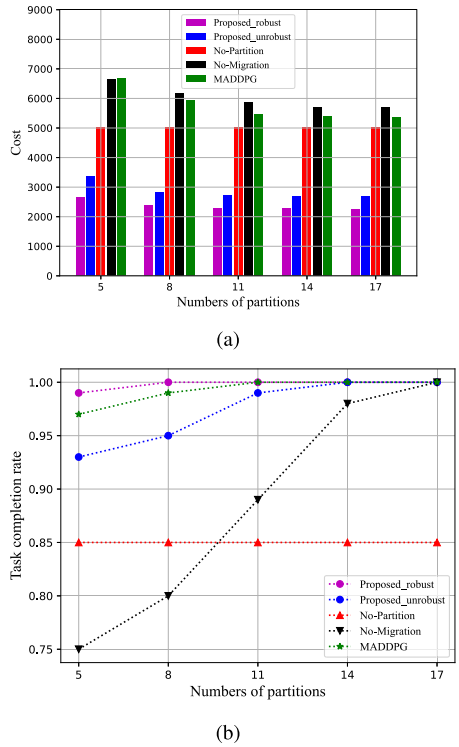


Fig. 9. The cost and task completion rate versus the numbers of partitions.

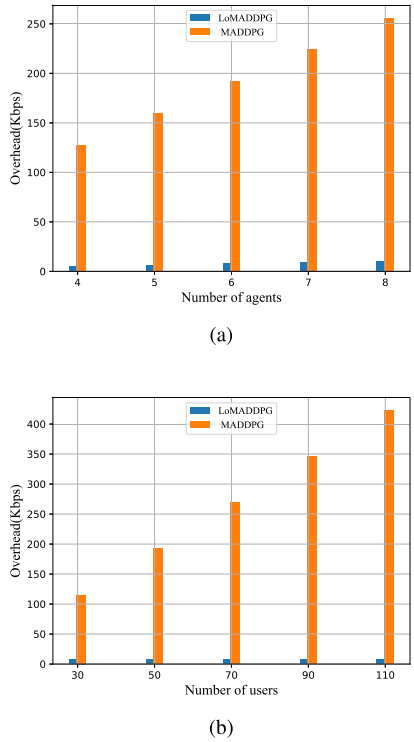


Fig. 10. Overhead comparisons of our LoMADDPG with MADDPG.

spans values from -1.0 to 100.0×10^6. Transmission of such float type values in the network requires conversion of each value to double-precision floating-point format which requires 64 bits for each value [34]. In our LoMADDPG method, relax the huge amount of state space transmission by only sending the weights of the agents' neural network (which are spanning from -1 to $+1$) only needs IEEE 754 half-precision binary conversion, i.e., 16 bits for each value. This difference is more pronounced when the number of agents and users as shown in Fig. 10. The number of BSs are 3, the number of RSUs controlled by each BS is 5.

Given its ability to solve optimal policies for agents in such environments, deep reinforcement learning is well-suited for addressing our resource allocation problem within the proposed system. DDPG is capable of acquiring policies within high-dimensional, continuous action and space environments. In addition, in the context of MA-DRL, having five actions per agent is quite manageable. Modern DRL algorithms are capable of handling significantly larger action spaces, especially with advancements in neural network architectures and training techniques. Although learning methods are capable of working with high-dimensional action spaces, in some cases, increasing the dimensions can slow down the convergence speed. One of the methods for reducing the dimensionality of the action space is the use of a hierarchical approach. Based on the hierarchical decision, each BS activates some RSUs in the large time slot so that the activated RSU serves its users in each small time slot. In a hierarchical approach, high-level policies decide which low-level policies (each handling subsets of actions) to execute. This approach effectively reduces the complexity each individual policy needs to manage. Another method is to discretize continuous actions or to merge multiple actions. The action spaces of our optimization problem, almost represent a distinct, discrete choice. Q_u is the number of partitions into which the task of user u can be divided, determined by each user u. $\check{\rho}^{q_u}_{u,b}(t_\tau)$ indicates whether user u processes the q-th portion of its task locally or not. This variable is determined by each user u. $\rho^{\kappa q_u}_{ui,b}(t_\tau)$ indicates whether the user u offloads it's q-th portion of task to agent i on shubcarrier κ or not. It has $Q_u \times N$ dimension where N is the total number of subcarriers. $\tilde{\rho}^{\kappa q_u}_{ui,b}(t_\tau)$ indicates whether the q-th portion of task of user u is only relayed by node i controlled by BS b on relative subcarrier at short time-scale slot. It has $N \times Q_u \times (R + B + V + P)$ dimension, where V, P, R, and B are the total number of vehicles, pedestrians, RSUs, and BSs, respectively. It has $N \times Q_u \times (R + B + V + P)$ dimension. $\mu^{\kappa \bar{q}_u}_{ij,b}(t_\tau)$ is used for migration decision with the same dimension. All of these actions are binary (with values of 0 and 1) which simplifies the action space significantly compared to continuous action spaces. The only continuous variable is $p^{\kappa}_{ij,b}(t_\tau)$. We can consider the uniform power distribution over subcarriers and remove this variable or discretize this variable into finite levels to accelerate the training process. To cope with a large number of possible actions, we consider the uniform power distribution over subcarriers and remove this variable from action space. Simulation result in Fig. 11 shows that by uniformly distributing the transmitted power across subcarriers, the training speed of the network can be significantly increased with only a slight reduction in performance.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.



Fig. 11. Average accumulated reward for proposed LoMADDPG with/without action space reduction.

VI. Conclusion

In this paper, we proposed a novel two-time scale task offloading/delivery framework tailored for users in a complex MEC environment in which we dynamically manage radio, storage, computing, and cost aspects through multiple MEC servers distributed across RSUs, BSs, vehicles, and pedestrians using task partitioning and migration. We also addressed the issue of demand uncertainty which arises from users' high mobility and traffic variations and I-CSI, by proposing a robust and cooperative two-time-scale task offloading/delivery approach in which we aimed at at minimizing task offloading and delivery costs while maximizing task completion rates. To efficiently implement this framework, we proposed the LoMADDPG method. Our result showed that while the existing MADDPG method performs slightly better in terms of task completion rate, huge saving is achieved in the task offloading cost by drastically decreasing the communication overhead between agents. Nevertheless, when we take demand uncertainty into account, the robust LoMADDPG method when trained properly, outperforms MADDPG in terms of task completion rate as well.

Acknowledgment

The content is solely the responsibility of the authors and does not necessarily represent the official views of QRDI.

References

[1] Y. Li, L. J. Cimini, and N. R. Sollenberger, "Robust channel estimation for OFDM systems with rapid dispersive fading channels," *IEEE Trans. Commun.*, vol. 46, no. 7, pp. 902–915, Jul. 1998.

[2] M. Chen et al., "Distributed learning in wireless networks: Recent progress and future challenges," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3579–3605, Dec. 2021.

[3] P. Chennakesavula, J.-M. Wu, and A. Ambikapathi, "Incentive-driven fog-edge computation offloading and resource allocation for 5G-NR V2X-based vehicular networks," in *Proc. IEEE Veh. Technol. Conf.*, Florence, Italy, Jun. 2023, pp. 1–5.

[4] S. Wang, R. Urgaonkar, M. Zafer, T. He, K. Chan, and K. K. Leung, "Dynamic service migration in mobile edge computing based on Markov decision process," *IEEE/ACM Trans. Netw.*, vol. 27, no. 3, pp. 1272–1288, Jun. 2019.

[5] K. Ali et al., "Raven: Vision-based connected vehicle safety platform using infrastructure sensing, 5G, and MEC," *IEEE Trans. Veh. Technol.*, early access, Jul. 19, 2024, doi: 10.1109/TVT.2024.3430815.

[6] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on Non-IID data with reinforcement learning," in *Proc. IEEE Conf. Comput. Commun.*, Toronto, ON, Canada, Jul. 2020, pp. 1698–1707.

[7] L. Zhao et al., "MESON: A mobility-aware dependent task offloading scheme for urban vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4259–4272, May 2024.

[8] S. Li, W. Sun, Q. Ni, and Y. Sun, "Road side unit-assisted learning-based partial task offloading for vehicular edge computing system," *IEEE Trans. Veh. Technol.*, vol. 73, no. 4, pp. 5546–5555, Apr. 2023.

[9] H. Li, X. Li, M. Zhang, and B. Ulziinyam, "System-wide energy efficient computation offloading in vehicular edge computing with speed adjustment," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 2, pp. 701–715, Jun. 2024.

[10] R. Huang, W. Wen, X. Chen, Z. Zhou, Q. Chen, and C. Dong, "Joint power allocation and task replication for reliability-sensitive services in NOMA-enabled vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 73, no. 3, pp. 4178–4193, Mar. 2024.

[11] C. Liu and K. Liu, "Toward reliable DNN-based task partitioning and offloading in vehicular edge computing," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 3349–3360, Feb. 2024.

[12] D. Wang, H. Zhu, C. Qiu, Y. Zhou, and J. Lu, "Distributed task offloading in cooperative mobile edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 7, pp. 10487–10501, Jul. 2024.

[13] Q. Liu, H. Zhang, X. Zhang, and D. Yuan, "Improved DDPG based two-timescale multi-dimensional resource allocation for multi-access edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 9153–9158, Jun. 2024.

[14] S. B. Azmy, N. Zorba, and H. S. Hassanein, "Incentive-vacation queueing for edge crowd computing," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 13167–13179, Apr. 2024.

[15] T. Shi, T. Zhang, R. Zhong, Y. Liu, and R. Huang, "Cross-user dependent task offloading and resource allocation in spatial-temporal dynamic MEC networks," *IEEE Trans. Veh. Technol.*, vol. 73, pp. 15584–15597, Oct. 2024.

[16] N. Fofana, A. B. Letaifa, and A. Rachedi, "Intelligent task offloading in vehicular networks: A deep reinforcement learning perspective," *IEEE Trans. Veh. Technol.*, early access, Jul. 25, 2024, doi: 10.1109/TVT.2024.3427814.

[17] Y. Li, L. Li, and P. Fan, "Mobility-aware computation offloading and resource allocation for NOMA MEC in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 8, pp. 11934–11948, Aug. 2024.

[18] H. Liu, W. Huang, D. I. Kim, S. Sun, Y. Zeng, and S. Feng, "Towards efficient task offloading with dependency guarantees in vehicular edge networks through distributed deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 13665–13681, Sep. 2024.

[19] Z. Niu, Y. Wu, J. Gong, and Z. Yang, "Cell zooming for cost-efficient green cellular networks," *IEEE Commun. Mag.*, vol. 48, no. 11, pp. 74–79, Nov. 2010.

[20] X. Chen, "Decentralized computation offloading game for mobile cloud computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 4, pp. 974–983, Apr. 2015.

[21] J. Crichigno, D. Oliveira, M. Pourvali, N. Ghani, and D. Torres, "A routing and placement scheme for network function virtualization," in *Proc. IEEE Int. Conf. Telecommun. Signal Process.*, Barcelona, Jul. 2017, pp. 26–31.

[22] 3GPP, "Evolved universal terrestrial radio access (E-UTRA); further advancements for E-UTRA physical layer aspects," 3rd Generation Partnership Project (3GPP), Tech. Rep. 36.814, 3, 2017, version 9.2.0. [Online]. Available: http://www.3gpp.org/DynaReport/36814.htm

[23] J. Karedal, N. Czink, A. Paier, F. Tufvesson, and A. F. Molisch, "Path loss modeling for Vehicle-to-Vehicle communications," *IEEE Trans. Veh. Technol.*, vol. 60, no. 1, pp. 323–328, Jan. 2011.

[24] X. Lyu, H. Tian, C. Sengul, and P. Zhang, "Multiuser joint task offloading and resource optimization in proximate clouds," *IEEE Trans. Veh. Technol.*, vol. 66, no. 4, pp. 3435–3447, Apr. 2017.

[25] S. Guo and X. Zhou, "Robust resource allocation with imperfect channel estimation in NOMA-based heterogeneous vehicular networks," *IEEE Trans. Commun.*, vol. 67, no. 3, pp. 2321–2332, Mar. 2019.

[26] Y. Chen, M. Wen, L. Wang, W. Liu, and L. Hanzo, "SINR-outage minimization of robust beamforming for the non-orthogonal wireless downlink," *IEEE Trans. Commun.*, vol. 68, no. 11, pp. 7247–7257, Nov. 2020.

[27] Z. Zhu, S. Wan, P. Fan, and K. B. Letaief, "Federated multi-agent actor-critic learning for age sensitive mobile edge computing," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1053–1067, Jan. 2022.

[28] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2015, p. 13.

[29] M. Sipper, "A serial complexity measure of neural networks," in *Proc. IEEE Int. Conf. Neural Netw.*, San Francisco, USA, Mar. 1993, pp. 962–966.

[30] X. He, H. Lu, M. Du, Y. Mao, and K. Wang, "QoE-based task offloading with deep reinforcement learning in edge-enabled internet of vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2252–2261, Apr. 2020.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.

[31] X. Zhang, J. Zhang, Z. Liu, Q. Cui, X. Tao, and S. Wang, "MDP-based task offloading for vehicular edge computing under certain and uncertain transition probabilities," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3296–3309, Mar. 2020.
[32] Y. Wang, W. Fang, Y. Ding, and N. Xiong, "Computation offloading optimization for UAV-assisted mobile edge computing: A deep deterministic policy gradient approach," *Wireless Netw.*, vol. 27, no. 4, pp. 2991–3006, May 2021.
[33] J. Wu and H. Li, "Deep ensemble reinforcement learning with multiple deep deterministic policy gradient algorithm," *Math. Problems Eng.*, vol. 2020, no. 1, Mar. 2020, Art. no. 4275623.
[34] W. Kahan, "IEEE standard 754 for binary floating-point arithmetic," Lecture Notes on the Status of IEEE, Elect. Eng. Computer Science, University of California, vol. 754, no. 94720-1776, p. 11, 1996.



Mohammad Reza Abedi (Student Member, IEEE) received the M.Sc. degree in electrical engineering from AmirKabir University, Tehran, Iran. He is currently working toward the Ph.D. degree with Tarbiat Modares University, Tehran. He is currently a Research Assistant with Tarbiat Modares University. He has been involved in a number of large scale network design and consulting projects in the telecom industry as a principle investigator or consultant. His research interests include multiple access techniques, energy harvesting and wireless power transfer, cooperative and adaptive wireless communications, wireless edge caching, mobile edge computing, multibitrate video transcoding, software defined networking, wireless network virtualization, and optimization theory. He was a Reviewer for several IEEE journals such as, IEEE TRANSACTIONS ON SIGNAL PROCESSING and IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS. He was also a Member of Technical Program Committees for the IEEE Conferences.



Nader Mokari (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from Tarbiat Modares University, Tehran, Iran, in 2014. His thesis received the IEEE outstanding Ph.D. thesis award. In 2015, he joined the Department of Electrical and Computer Engineering, Tarbiat Modares University as an Assistant Professor. He is currently an Associated Professor with the Department of Electrical and Computer Engineering, Tarbiat Modares University, Tehran, Iran. His research interests focuses on many aspects of wireless technologies with a special emphasis on wireless networks. He has been elected as an IEEE exemplary reviewer, in 2016 by IEEE Communications Society. Dr. Mokari is on the Editorial Board of IEEE TRANSACTIONS ON COMMUNICATIONS. In recent years, his research has been funded by Iranian Mobile Telecommunication Companies, Iranian National Science Foundation. He was the recipient of Best Paper Award with ITU K-2020. He was also involved in a number of large scale network design and consulting projects in the telecom industry.



Mohammad Reza Javan (Senior Member, IEEE) received the B.Sc. degree in electrical engineering from Shahid Beheshti University, Tehran, Iran, in 2003, the M.Sc. degree in electrical engineering from the Sharif University of Technology, Tehran, Iran, in 2006, and the Ph.D. degree in electrical engineering from Tarbiat Modares University, Tehran, Iran, in 2013. He is currently a Faculty Member with the Faculty of Electrical Engineering, Shahrood University of Technology, Shahrood, Iran. His research interests include design and analysis of wireless communication networks with emphasis on the application of optimization theory, and machine learning methods.



Hamid Saeedi (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees in electrical engineering from the Sharif University of Technology, Tehran, Iran, in 1999 and 2001, respectively, and the Ph.D. degree in electrical engineering from Carleton University, Ottawa, ON, Canada, in 2007. From 2008 to 2009, he was a Postdoctoral Fellow with the Department of Electrical and Computer Engineering, University of Massachusetts, Amherst, MA, USA. In 2010, he joined the Department of Electrical and Computer Engineering, Tarbiat Modares University, Tehran, Iran. Since 2021, he has been with the College of Engineering, University of Doha for Science and Technology, Doha, Qatar as an Associate Professor. His research interests focuses on broad area of communications. He was a Member of the Editorial Board of IEEE COMMUNICATIONS LETTERS and IEEE TRANSACTIONS ON COMMUNICATIONS from 2014 to 2018 and from 2018 to 2020, respectively.



Eduard A. Jorswieck (Fellow, IEEE) received the Ph.D. degree in electrical engineering and computer science from, Technical University Berlin Berlin, Germany, in 2004. From 2006 to 2008, he was with the Signal Processing Group, KTH Royal Institute of Technology, Stockholm, Sweden, as a Postdoctoral Fellow and an Assistant Professor. From 2008 to 2019, he was the Chair for communication theory with TU Dresden, Dresden, Germany. He is currently the Managing Director of the Institute of Communications Technology, the Head of the Chair for Communications Systems, and a Full Professor with Technische Universität Braunschweig, Brunswick, Germany. He has authored or coauthored more than 190 journal articles, 15 book chapters, one book, three monographs, and some 330 conference papers. His main research focuses on communications. He was the recipient of the IEEE Signal Processing Society Best Paper Award. He and his colleagues were also the recipient of the Best Paper and Best Student Paper Awards at the IEEE CAMSAP 2011, IEEE WCSP 2012, IEEE SPAWC 2012, IEEE ICUFN 2018, PETS 2019, and ISWCS 2019. Since 2017, he has been the Editor-in-Chief of the *EURASIP Journal on Wireless Communications and Networking*. He is currently serving on the editorial boards of the IEEE TRANSACTIONS ON INFORMATION THEORY and IEEE TRANSACTIONS ON COMMUNICATIONS. He was on the Editorial Boards of the IEEE SIGNAL PROCESSING LETTERS, the IEEE TRANSACTIONS ON SIGNAL PROCESSING, the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, and the IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY.



Nizar Zorba (Senior Member, IEEE) received the B.Sc. degree in electrical engineering from the Jordan University of Science and Technology, Jordan, in 2002, and the Ph.D. degree in signal processing for communications from the Universitat Politècnica de Catalunya Barcelona Tech - UPC, Barcelona, Spain, in 2007. He is currently a Professor with the Electrical Engineering Department, Qatar University, Doha, Qatar. He has authored six international patents, two books, contributed to seven book chapters, and coauthored more than 180 papers in peer-reviewed journals and international conferences. He is an Associate EiC of the *IEEE Communications Magazine*, an Area Editor of the IEEE COMMUNICATIONS LETTERS and an Associate Editor of IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING. He is also an IEEE ComSoc Distinguished Lecturer. Dr. Zorba is a Former Chair of the IEEE Communication Society Technical Committee on Communications Systems Integration and Modeling.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:40:10 UTC from IEEE Xplore. Restrictions apply.